

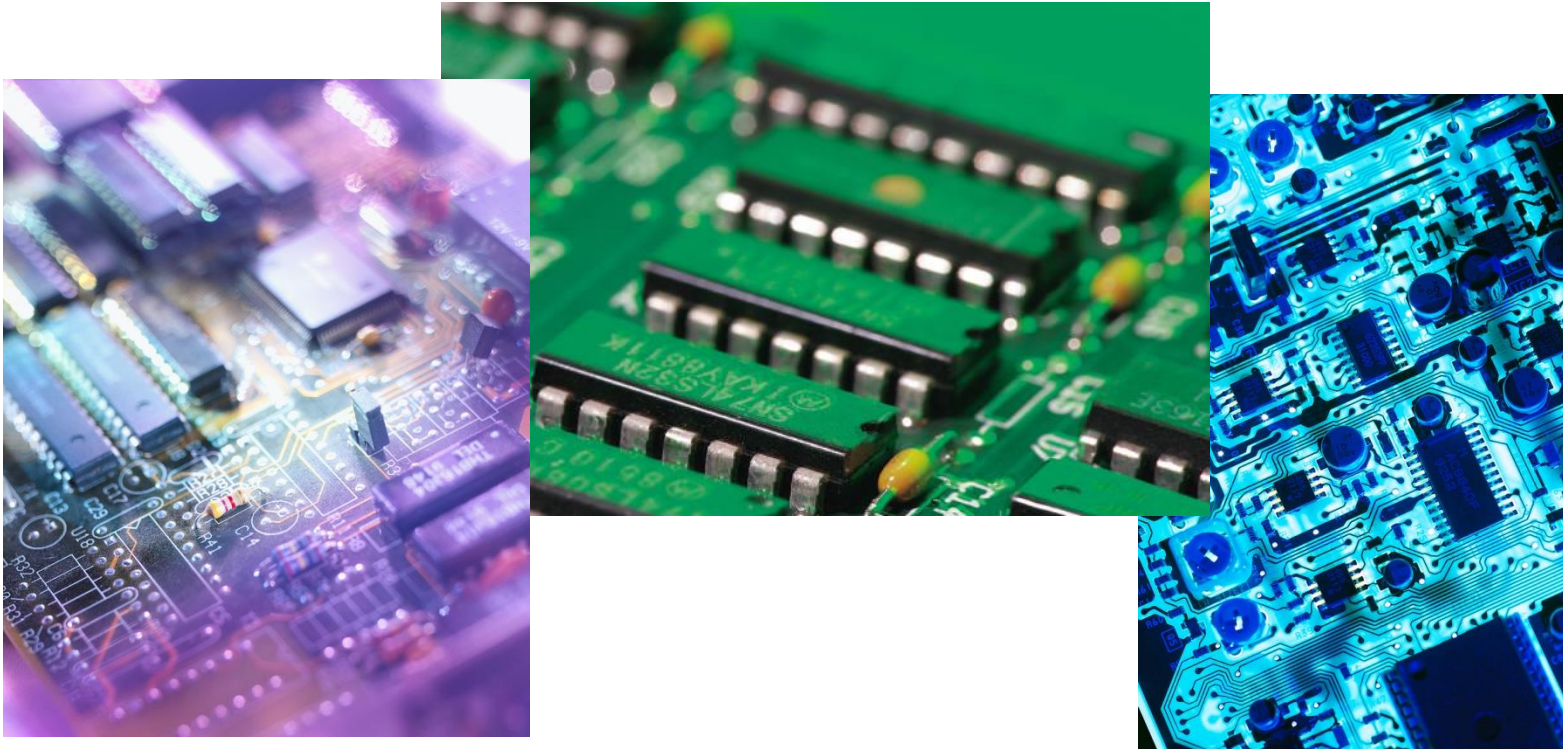
第2章 并行硬件与并行软件

深圳大学计算机与软件学院 陆克中

路线图

- 背景知识
- 冯·诺伊曼模型的改进
- 并行硬件
- 并行软件
- 输入和输出
- 性能
- 并行程序设计
- 编写和运行并行程序

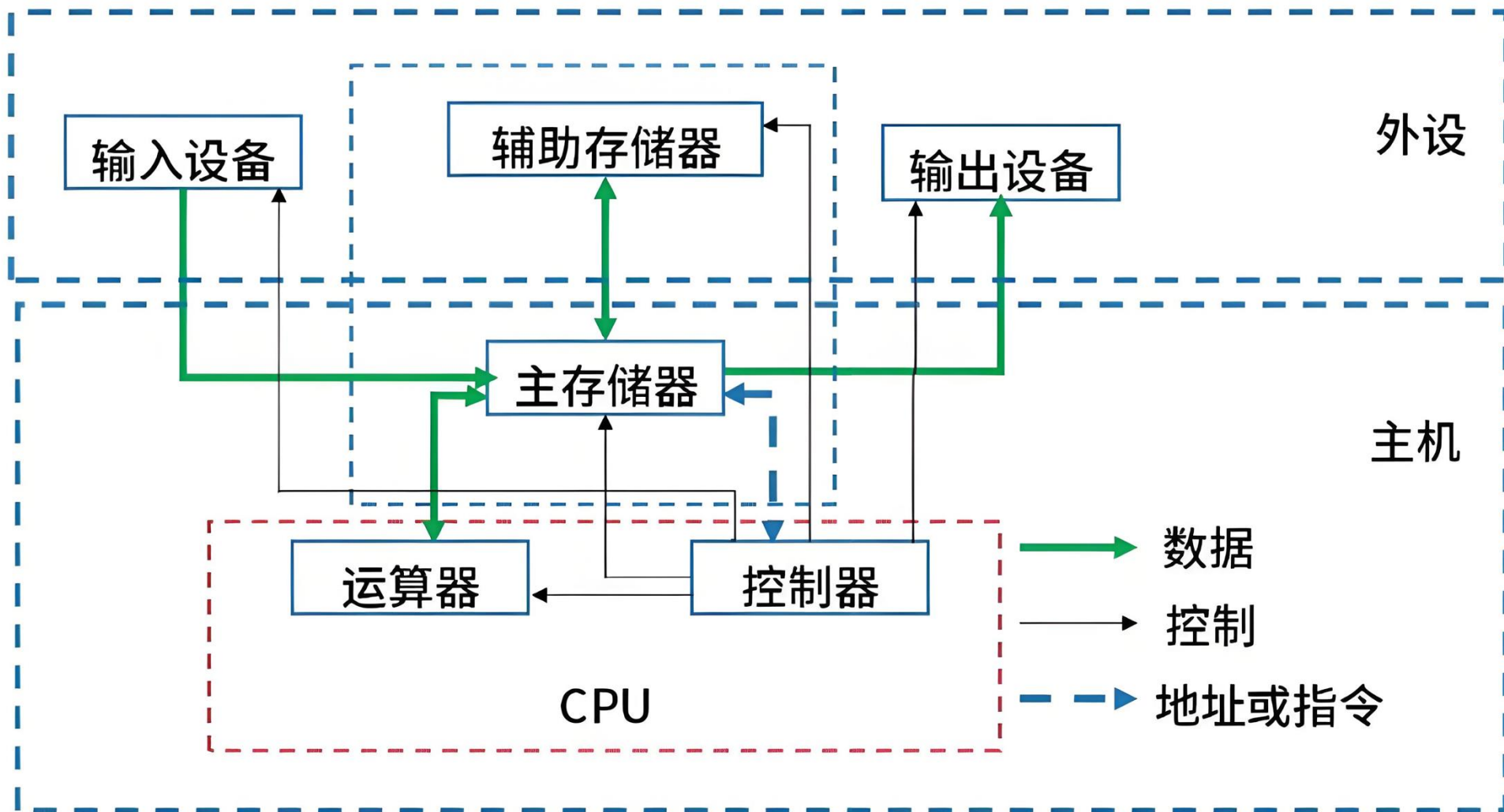
背景知识



串行硬件和软件



冯·诺依曼体系结构



主存储器

- 由一组位置组成，每个位置都能存储指令和数据。
- 每个位置都有一个地址及其存储的内容。
 - ▶ 地址用于访问该位置，该位置的内容是存储在该位置中的指令或数据。



中央处理器 (CPU)

■ 控制单元 (CU)

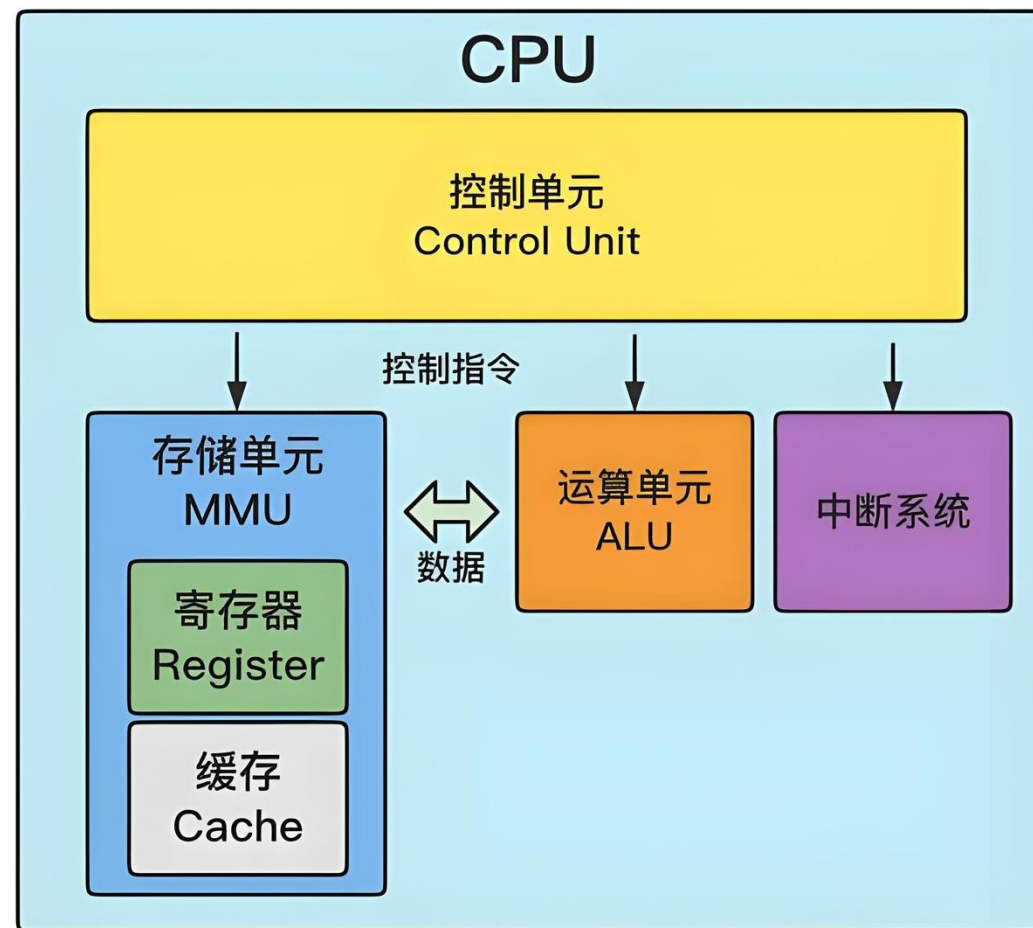
- ▶ 指挥、协调 CPU 内部及整个计算机的操作，从内存取指令、译码、发控制信号。

■ 运算单元 (ALU)

- ▶ 数据通路
- ▶ 负责算术运算（加减乘除）和逻辑运算（与、或、非、比较）。

■ 寄存器 (Register)

- ▶ CPU内部高速临时存储单元，存放当前正在处理的数据和指令。



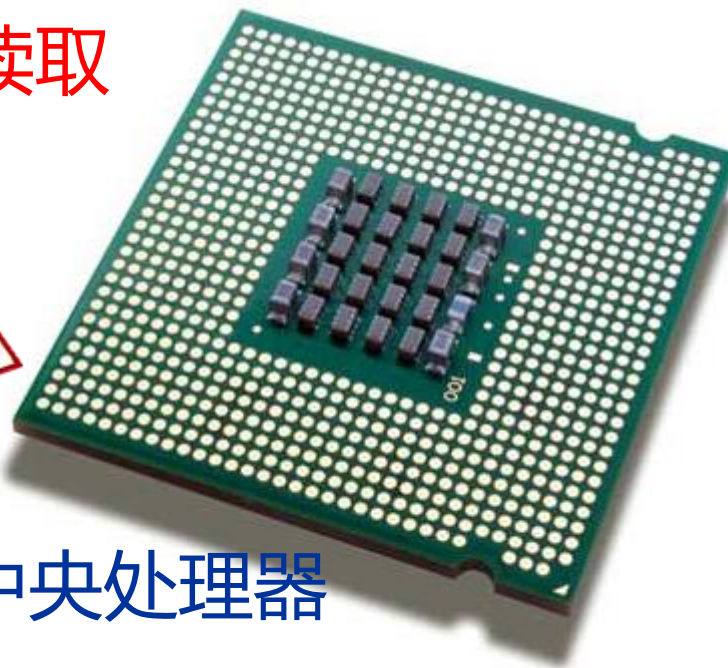
内存



提取/读取



中央处理器



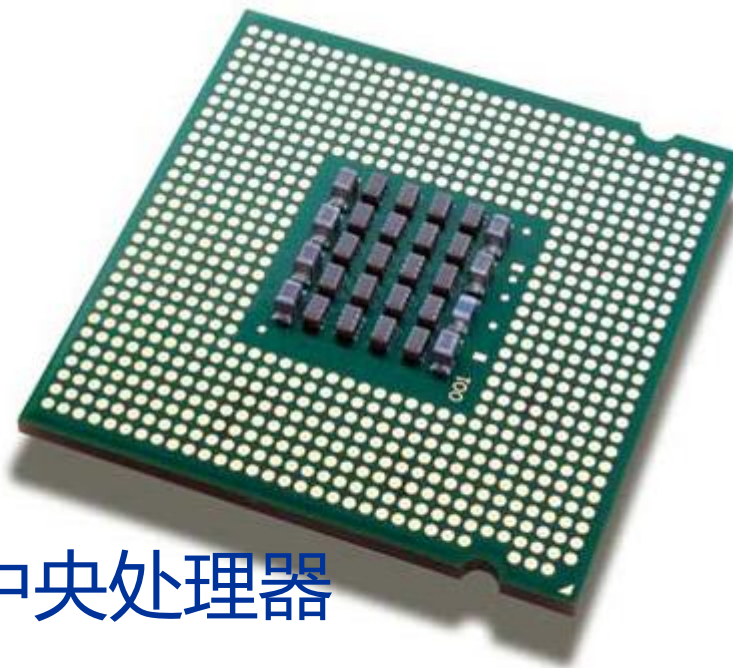
内存



写入/存储

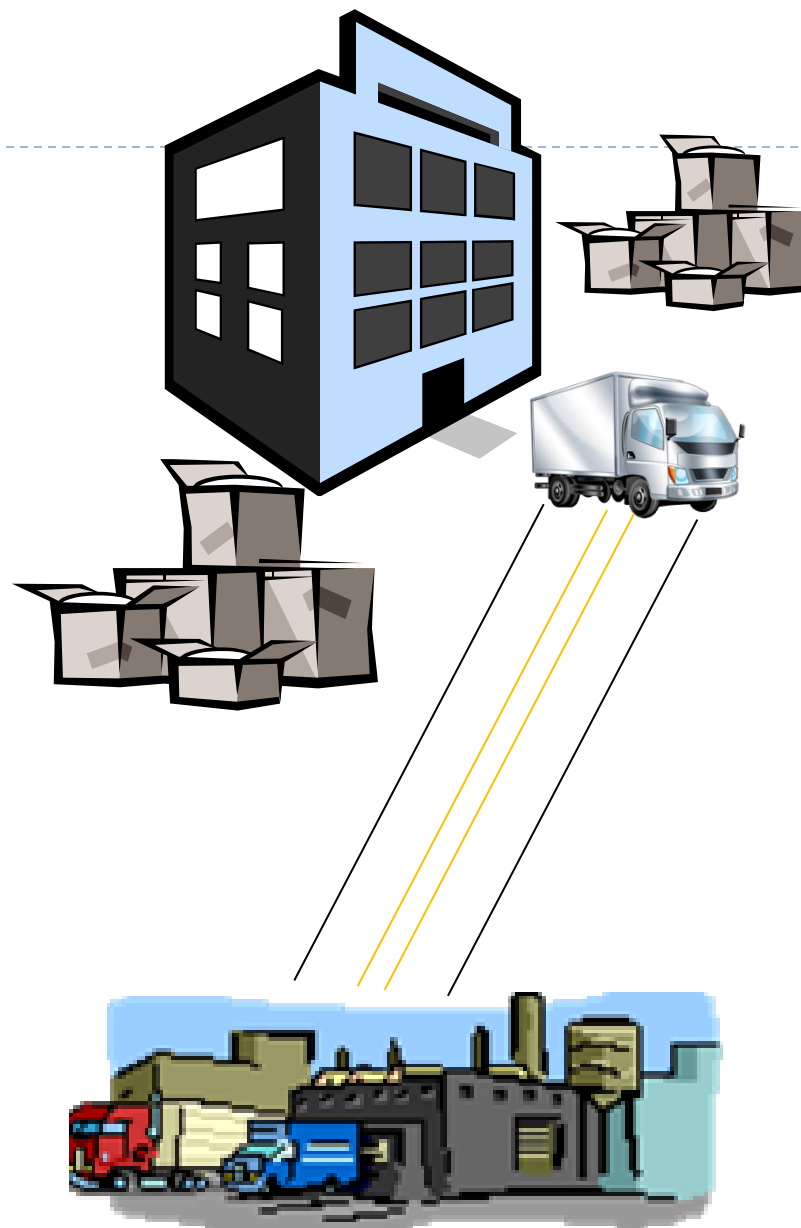


中央处理器



冯·诺依曼瓶颈

- 处理器与内存物理分离
- 数据必须通过总线来回搬运
- 计算速度远超数据传输速度
- 处理器长期等待数据
- 算力无法充分释放
- 内存墙

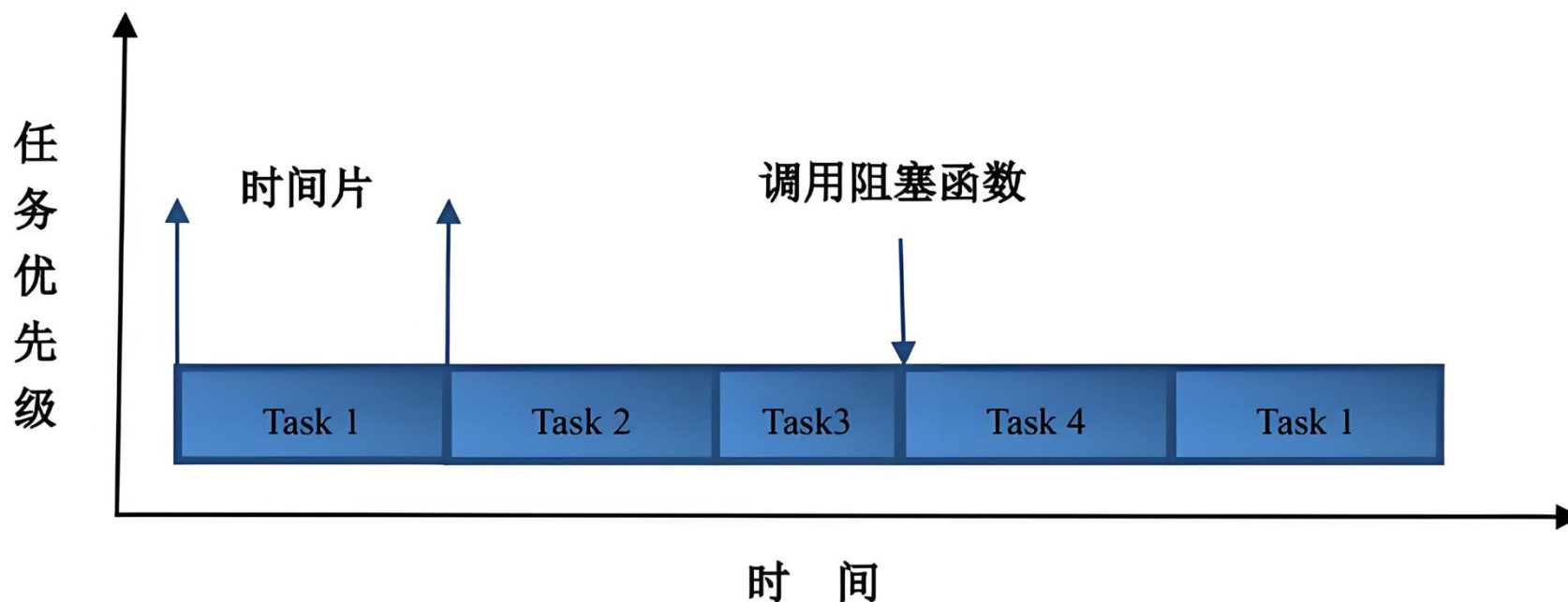


进程

- 正在执行的计算机程序的一个实例。
- 进程的组成部分：
 - ▶ 可执行的机器语言程序。
 - ▶ 内存块。
 - ▶ 操作系统分配给进程的资源描述符。
 - ▶ 安全信息。
 - ▶ 关于进程状态的信息。

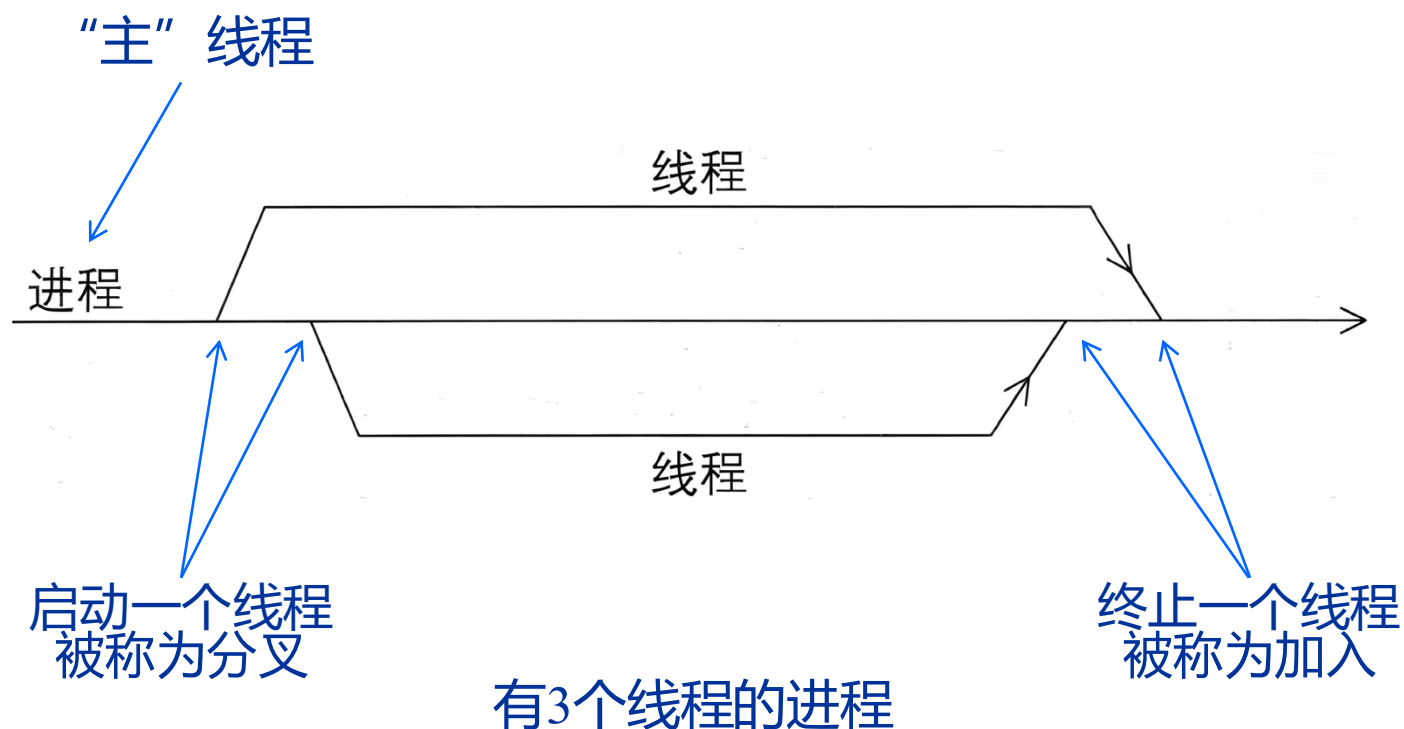
多任务

- 支持多个程序同时执行，即使在只有一个核心的系统上。
- 每个进程只运行一小段时间（通常为几毫秒），称为时间片。
- 正在运行的程序执行了一个**时间片**后，操作系统可以运行另一个程序。

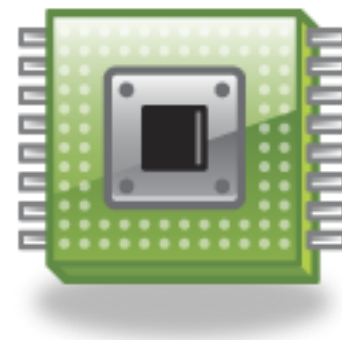


线程

- 线程包含在进程中。
- 可以将程序划分为或多或少独立的任务。
- 当一个线程被阻塞时，可以运行另一个线程。

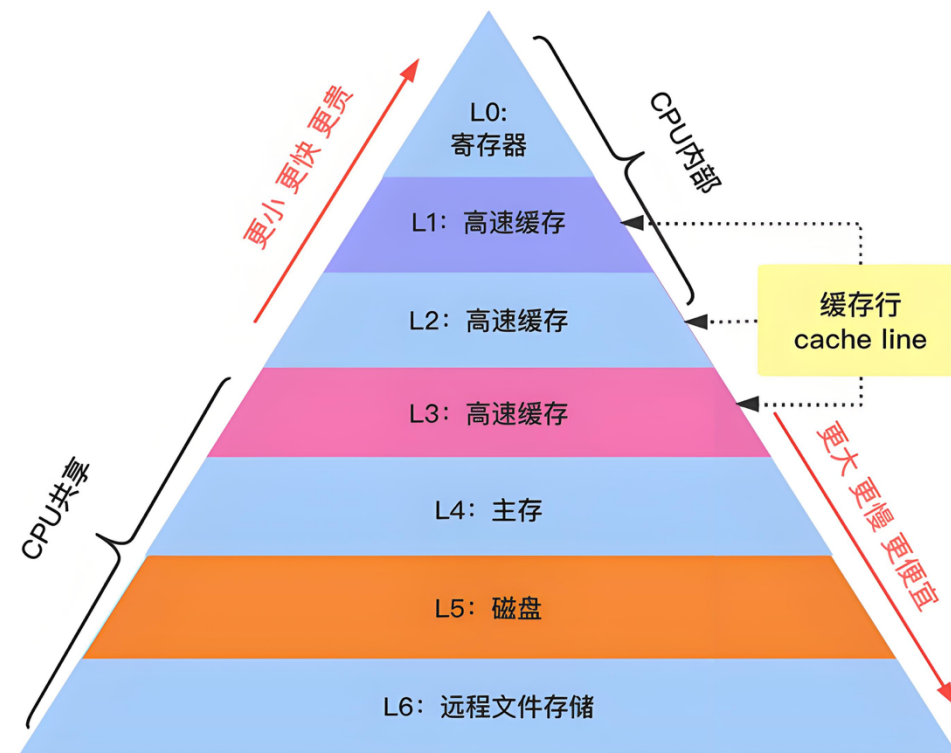


冯·诺伊曼模型的改进



缓存基础

- 一组内存位置的集合，相比其他内存位置，其访问时间更短。
- CPU缓存通常位于与CPU相同的芯片上。



缓存	Register	L1 Cache	L2 Cache	L3 Cache	Main Memory	硬盘
访问时间	<1ns	约1ns	约3ns	约15ns	约80ns	约2ms
典型容量	几十~几百B	几十~几百KB	几百KB	几百KB~几MB	几GB	几百GB~几TB

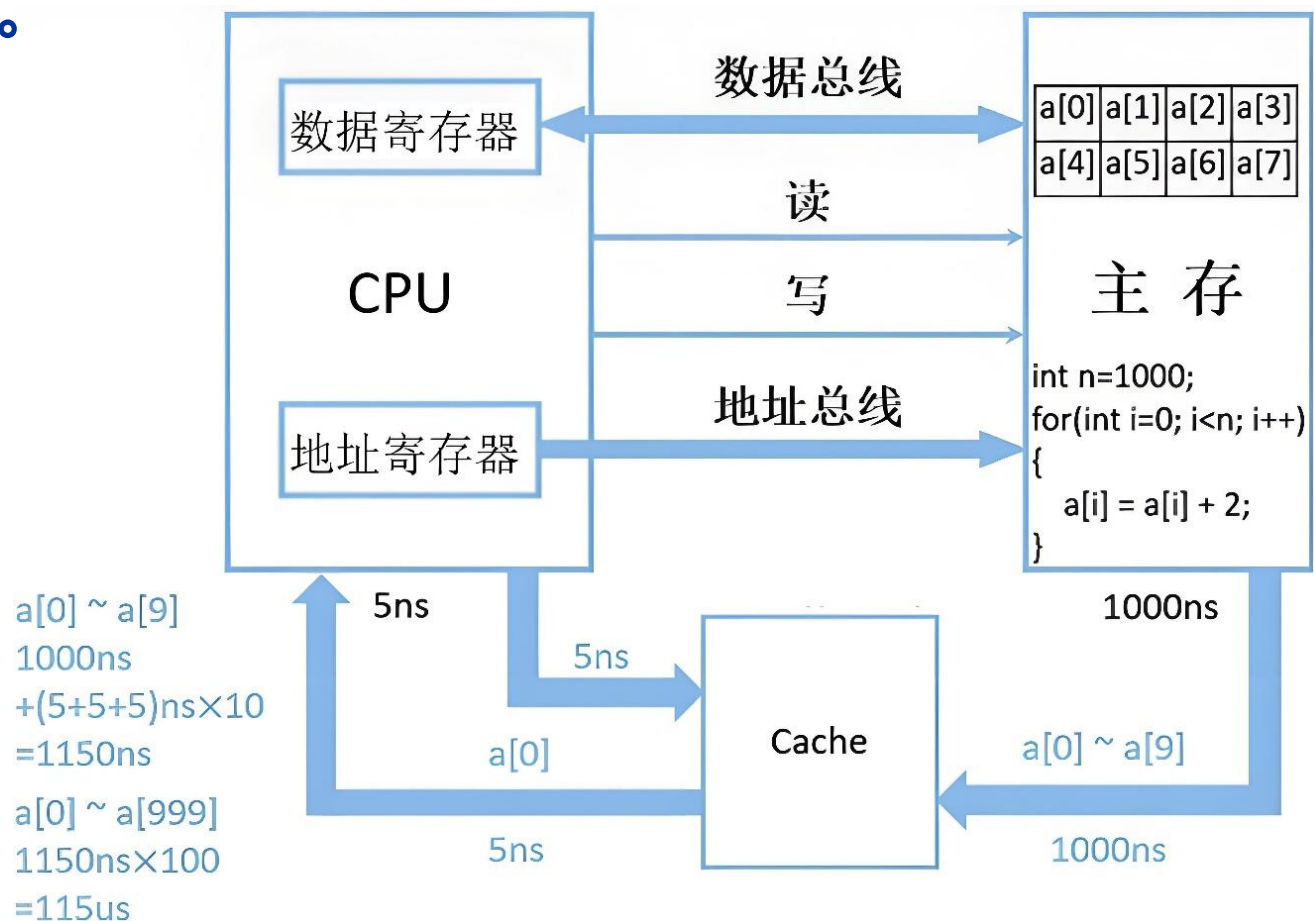
局部性原理

- 访问一个位置后再访问附近位置的原则。
- **空间局部性** - 访问附近的位置。
- **时间局部性** - 近期访问。

```
float z[1000];  
...  
sum = 0.0;  
for (i = 0; i < 1000; i++)  
    sum += z[i];
```

缓存命中和缓存缺失

- **缓存行** - CPU缓存中最小的数据读写与传输单位，主流大小为64字节。



空间局部性：在最近的未来要用到的信息(指令和数据)，很可能与现在正在使用的信息在存储空间上是邻近的

时间局部性：在最近的未来要用到的信息，很可能是现在正在使用的信息

仅考虑 $a[i] = a[i] + 2$
取出 $a[i]$ 耗时1000ns, 加法运算5ns,
存回 $a[i]$ 耗时1000ns, 故每个 $a[i]$ 耗
时2005ns, 1000个 $a[i]$ 共耗时
 $2005\text{ns} \times 1000 = 2005\mu\text{s}$

缓存问题

- 当CPU将数据写入缓存时，缓存中的值可能与主内存中的值不一致。
 - 。
- **写直达缓存** - 同时写入缓存 + 主存。
 - ▶ 优点：数据一致，简单。
 - ▶ 缺点：写慢，每次都要写内存。
- **写回缓存** - 只写缓存，标记为脏，替换时才写回主存。
 - ▶ 优点：写快。
 - ▶ 缺点：逻辑复杂，可能不一致。

缓存映射

- **全相联** - 新行可以放置在缓存中的任何位置。
- **直接映射** - 每个缓存行在缓存中都有唯一的位置。
- **n 路组相联** - 每个缓存行可以放置在缓存中 n 个不同位置之中的一个。

内存索引	缓存位置		
	全相联	直接映射	两路
0	0、1、2 或 3	0	0 或 1
1	0、1、2 或 3	1	2 或 3
2	0、1、2 或 3	2	0 或 1
3	0、1、2 或 3	3	2 或 3
4	0、1、2 或 3	0	0 或 1
5	0、1、2 或 3	1	2 或 3
6	0、1、2 或 3	2	0 或 1
7	0、1、2 或 3	3	2 或 3
8	0、1、2 或 3	0	0 或 1
9	0、1、2 或 3	1	2 或 3
10	0、1、2 或 3	2	0 或 1
11	0、1、2 或 3	3	2 或 3
12	0、1、2 或 3	0	0 或 1
13	0、1、2 或 3	1	2 或 3
14	0、1、2 或 3	2	0 或 1
15	0、1、2 或 3	3	2 或 3

缓存替换

- 当内存中的多个数据块可以映射到缓存中的多个不同位置（全相联和n路组相联）时，还需要决定应该替换或逐出缓存中的哪一行。
- 策略
 - ▶ FIFO - 先进先出，谁先来谁先走
 - ▶ LRU - 最近最少使用，最久不用就淘汰
 - ▶ LFU - 最不经常使用，用得最少就淘汰



缓存和程序

```
double A[MAX][MAX], x[MAX], y[MAX];
```

```
/* 初始化A和x, y赋值为0 */
```

```
/* 第一对循环 */
```

```
for (i = 0; i < MAX; i++)  
    for (j = 0; j < MAX; j++)  
        y[i] += A[i][j]*x[j];
```

```
/* y赋值为0 */
```

```
/* 第二对循环 */
```

```
for (j = 0; j < MAX; j++)  
    for (i = 0; i < MAX; i++)  
        y[i] += A[i][j]*x[j];
```

缓存行	A的元素			
0	A[0][0]	A[0][1]	A[0][2]	A[0][3]
1	A[1][0]	A[1][1]	A[1][2]	A[1][3]
2	A[2][0]	A[2][1]	A[2][2]	A[2][3]
3	A[3][0]	A[3][1]	A[3][2]	A[3][3]

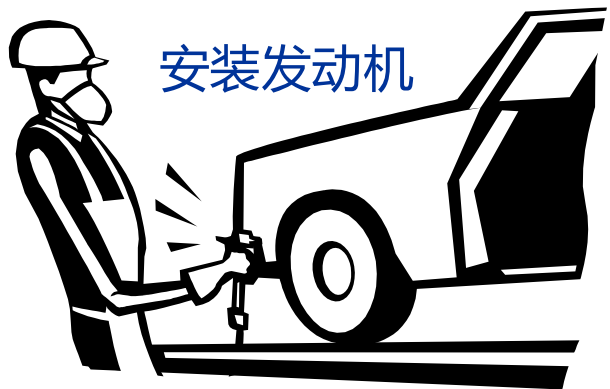
虚拟内存

- 如果运行一个非常大的程序或一个访问非常大的数据集的程序，那么所有的指令和数据可能都无法装入主内存。
- 虚拟内存充当辅助存储的缓存。
- 利用了时空局部性原理。
- 只在主存中保存许多正在运行的程序中的活跃部分。

指令级并行 (ILP)

- 尝试通过让多个处理器组件或功能单元同时执行指令来提高处理器性能。
- **流水线** - 功能单元分阶段排列。
- **多发射** - 可以同时启动多个指令。

流水线作业



流水线示例

时间	操作	操作数 1	操作数 2	结果
1	获取操作数	9.87×10^4	6.54×10^3	
2	比较指数	9.87×10^4	6.54×10^3	
3	移位一个操作数	9.87×10^4	0.654×10^4	
4	相加	9.87×10^4	0.654×10^4	10.524×10^4
5	标准化结果	9.87×10^4	0.654×10^4	1.0524×10^5
6	四舍五入结果	9.87×10^4	0.654×10^4	1.05×10^5
7	存储结果	9.87×10^4	0.654×10^4	1.05×10^5

将浮点数 9.87×10^4 和 6.54×10^3 相加

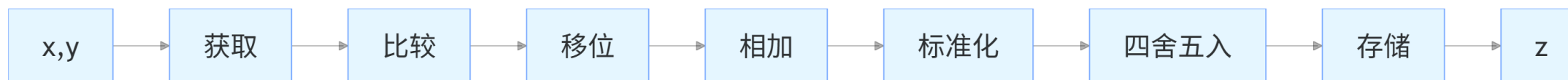
流水线示例

- 假设每次操作耗时1纳秒 (10^{-9} 秒)。
- 这个for循环大约需要7000纳秒。

```
float x[1000], y[1000], z[1000];  
.  
.  
.  
for (i = 0; i < 1000; i++)  
    z[i] = x[i] + y[i];
```

流水线

- 将浮点加法器划分为7个独立的硬件或功能单元。
 - ▶ 第1个单元获取两个操作数,
 - ▶ 第2个单元比较指数,
 - ▶
- 一个功能单元的输出是下一个功能单元的输入。



流水线

- 一次浮点加法仍然需要7纳秒。
- 但是1000次浮点加法现在只需要1006纳秒！

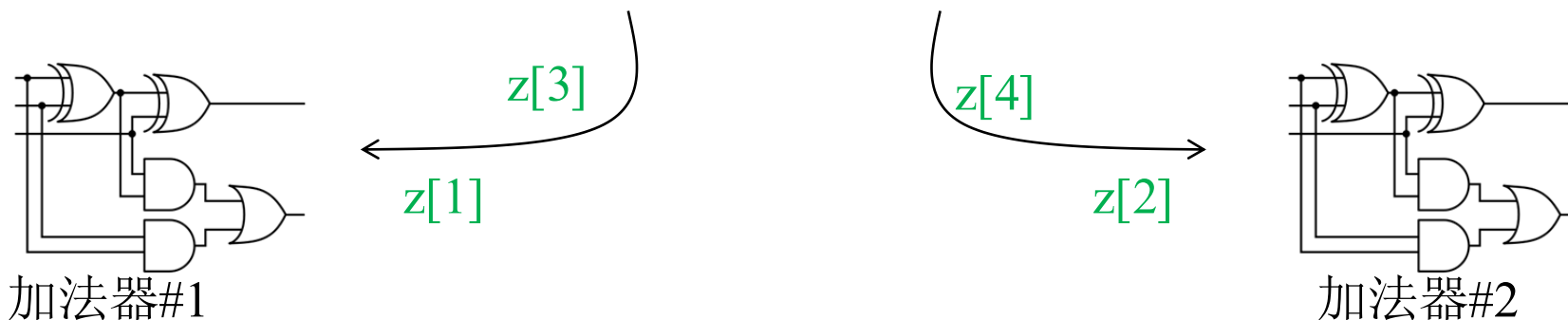
时间	获取	比较	移位	相加	标准化	四舍五入	存储
0	0						
1	1	0					
2	2	1	0				
3	3	2	1	0			
4	4	3	2	1	0		
5	5	4	3	2	1	0	
6	6	5	4	3	2	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
999	999	998	997	996	995	994	993
1 000		999	998	997	996	995	994
1 001			999	998	997	996	995
1 003					999	998	997
1 004						999	998
1 005							999

多发射

- 多发射处理器复制功能单元，并尝试在一个程序中同时执行不同的指令。

```
for (i = 0; i < 1000; i++)
```

```
    z[i] = x[i] + y[i];
```



多发射

- **静态多发射** - 由编译器提前安排好哪些指令可以并行发射。
- **动态多发射** - 由处理器硬件（运行时） 实时判断和安排指令的并行发射。
- **超标量处理器** - 支持动态多发射的处理器。

1. $a = x + y$

2. $b = a * 3$

3. $c = z - w$

// 编译器打包（静态多发射）

时钟周期1：指令1 + 指令3

时钟周期2：指令2

```
int a, b, res, x;
```

```
scanf("%d", &x);
```

```
//动态多发射：运行时识别无依赖，并行执行
```

```
if (x == 1) {
```

```
    a = 10 + 20;
```

```
}
```

```
else {
```

```
    b = 30 * 40;
```

```
}
```

```
res = 50 - 15;
```

前瞻执行

- 要利用多发射，系统必须找到可以同时执行的指令。
- 在前瞻执行中，编译器或处理器对指令进行猜测，然后根据猜测执行指令。

$z = x + y ;$

$\text{if} (z > 0)$

$w = x ;$

else

$w = y ;$



如果系统预测错误，则需要
退回并执行赋值 $w = y$ 。

硬件多线程

- 硬件多线程允许单个核心同时运行多个线程。
 - Hyper-Threading - 超线程
 - SMT - 同时多线程
- 当前线程暂停时（例如等待从内存读取数据），CPU切换运行另一个线程。
- 一个核心，多套寄存器，快速切换，假装多核，让CPU不闲着。

并行硬件

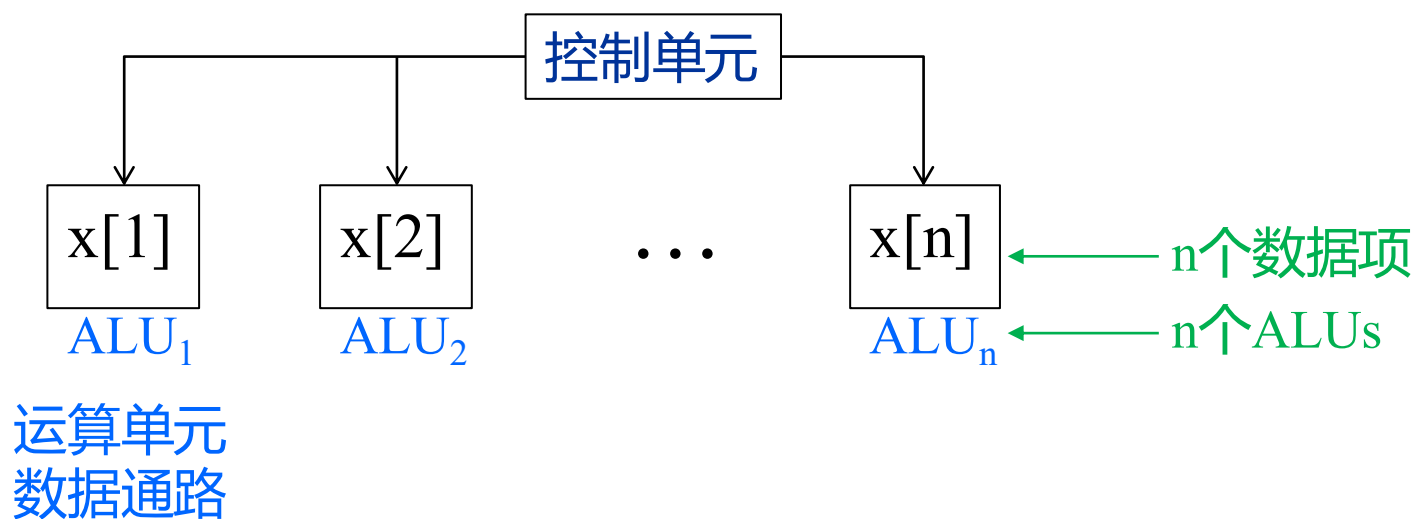


Flynn分类法

<i>经典冯·诺依曼结构</i> SISD 单指令流单数据流	(SIMD) 单指令流多数据流
MISD 多指令流单数据流 <i>现实中几乎没有</i>	(MIMD) 多指令流多数据流

SIMD

- 通过在处理器之间划分数据来实现并行处理。
- 将同一指令应用于多个数据项。
- 被称为数据并行。



```
for (i = 0; i < n; i++)  
    x[i] += y[i];
```

SIMD

- 如果 ALUs 数量少于数据项，该怎么办？
- 划分任务，迭代处理。
- 例如， $m = 4$ ALUs, $n = 15$ 个数据项。

Round	ALU1	ALU2	ALU3	ALU4
1	x[0]	x[1]	x[2]	x[3]
2	x[4]	x[5]	x[6]	x[7]
3	x[8]	x[9]	x[10]	x[11]
4	x[12]	x[13]	x[14]	

SIMD的缺点

- 所有数据通路执行相同指令或空闲。
- 在经典设计中，它们还必须同步运行。
- 数据通路没有指令存储区。
- 在处理大型数据并行问题时非常有效，但在处理其他类型的并行问题时往往表现不佳。

向量处理器

- 对数据的数组或向量进行操作，而传统的CPU则可以对单个数据元素或标量进行操作。
- 向量寄存器
 - ▶ 能够存储操作数向量，并同时对其内容进行操作。

类型	代表产品	向量位宽 / 长度	主要应用	架构特点
传统向量超级计算机	Cray-1、NEC SX	64-512位	科学计算、流体力学	寄存器 - 寄存器 / 内存 - 内存，专用流水线
x86 SIMD	AVX-512	512 位	通用计算、HPC、AI	固定位宽，掩码执行，兼容标量
ARM SIMD	NEON/SVE2	128位	移动、嵌入式、HPC	SVE 可扩展，适配多场景
RISC-V RVV	SiFive X390、NX27V	128-1024位	AI、边缘计算、HPC	开放标准，软件可配置
专用 AI 向量处理器	华为Ascend、GPU	数千并行通道	深度学习训练 / 推理	张量 / 向量融合，高并行密度

向量处理器

■ 向量化和流水线功能单元

- ▶ 相同的操作被应用于向量中的每个元素（或每对元素），SIMD。

■ 向量指令

- ▶ 对向量而不是标量进行操作。

■ 交错存储器

- ▶ 内存系统由多个可以或多或少独立访问的库组成。
- ▶ 如果一个向量的元素分布在多个库中，则加载 / 存储连续元素的延迟很小甚至没有延迟。

■ 跨步内存访问和硬件分散 / 聚集。

- ▶ 程序以固定的间隔访问向量的元素。

向量处理器 - 优点

- 使用快速且简单。
- 向量化编译器非常擅长识别可以向量化的代码。
- 编译器还可以提供关于循环无法向量化的原因的信息。
 - 帮助程序员重新评估代码。
- 非常高的内存带宽。
- 使用缓存行中的所有项。



向量处理器 - 缺点

- 不像其他并行体系结构那样处理不规则的数据结构。
- **可扩展性**有一个非常有限的限制，即处理更大问题的能力。



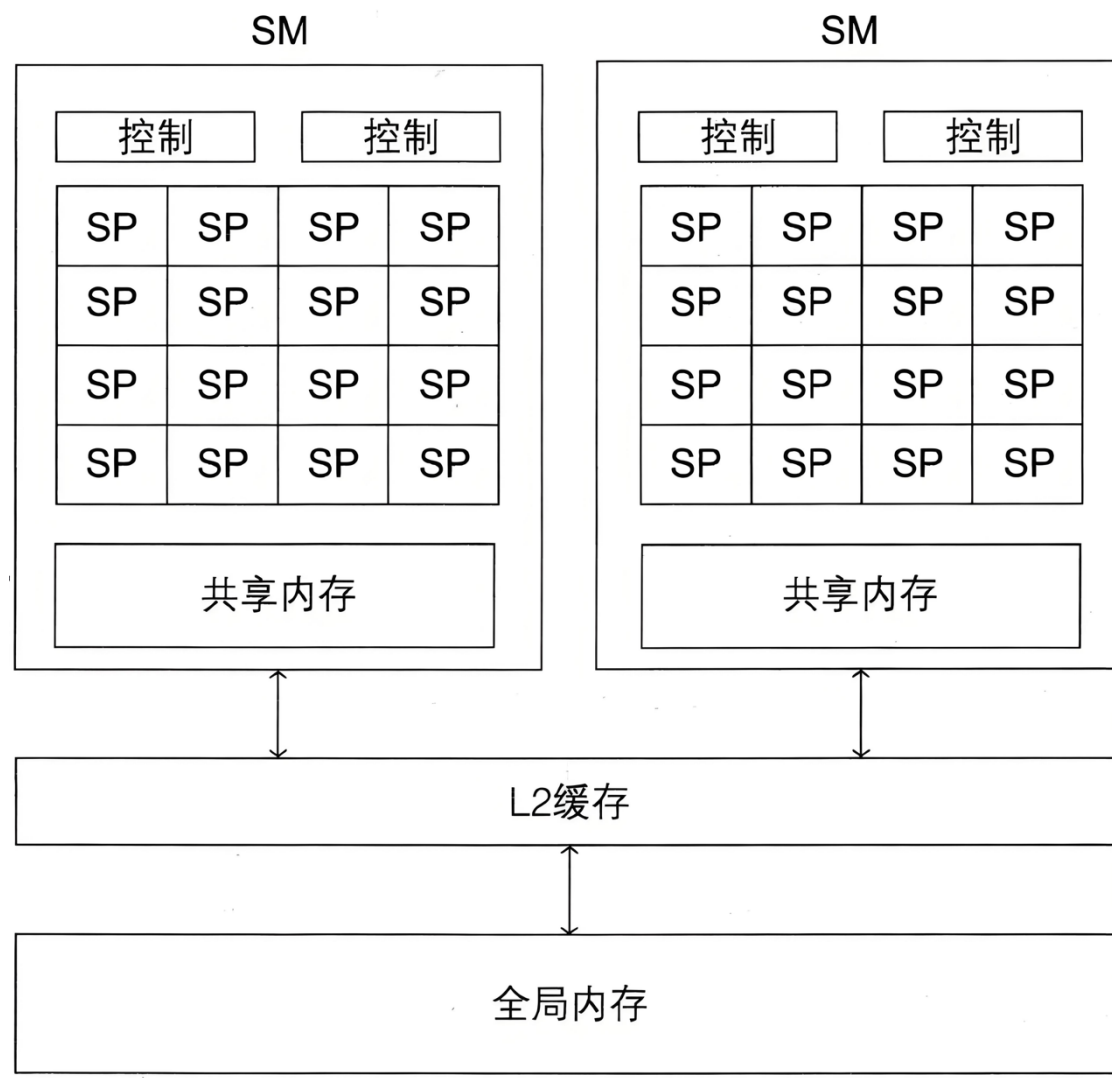
图形处理器

- 专门优化大规模并行图形与通用计算的微处理器，属于协处理器。
- 最初用于2D/3D图形渲染，后通过 GPGPU（通用图形处理器）技术拓展至非图形高性能计算。
- 使用SIMD并行，但不是纯SIMD系统。

维度	CPU	GPU
核心数量	少量强核心（常见 4-32 核）	数千至数万弱核心
擅长任务	复杂串行、分支预测、逻辑控制	大规模并行、重复简单运算
缓存 / 带宽	多级大容量缓存，内存带宽一般	精简缓存，极高显存带宽
典型场景	操作系统、办公软件、复杂逻辑程序	游戏渲染、AI 训练、影视特效、科学计算

图形处理器

- GPU由流式多处理器（SM）组成，一个SM可以有多个控制单元和更多数据通路。
 - ▶ 可将SM视为由一个或多个SIMD处理器组成。
 - ▶ 数据通路被称为核心或流处理器（SP）。
- NVIDIA GeForce RTX 4090D有114个SM，每个SM有128个SP，总共有14592个SP。

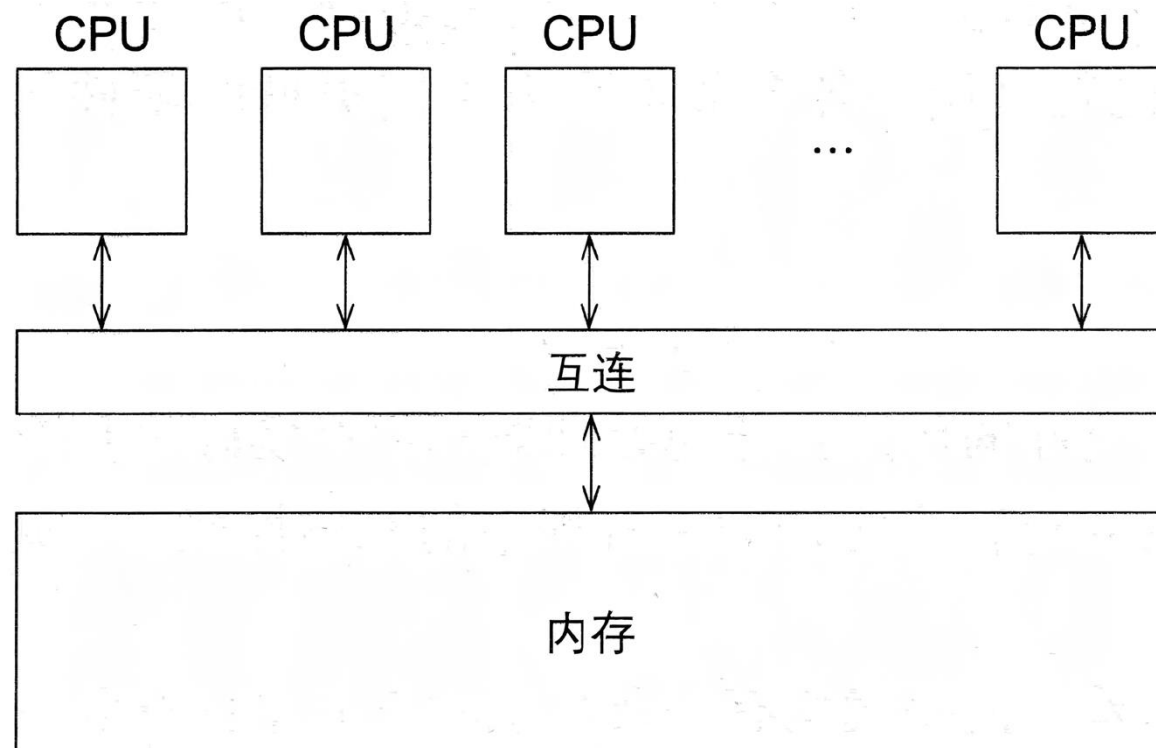


MIMD

- 支持在多个数据流上运行多个并行指令流。
- 通常由完全独立的处理单元或核心组成，每个处理单元或核心都有自己的控制单元和数据通路。
- 异步运行，通常没有全局时钟。

共享内存系统

- 一组自主处理器通过互连网络连接到内存系统。
- 每个处理器可以访问每个内存位置。
- 处理器通常通过访问共享数据结构来进行隐式通信。



共享内存系统

- 最广泛使用的共享内存系统使用一个或多个多核处理器。
 - ▶ 在单个芯片上有多个CPU或核心。



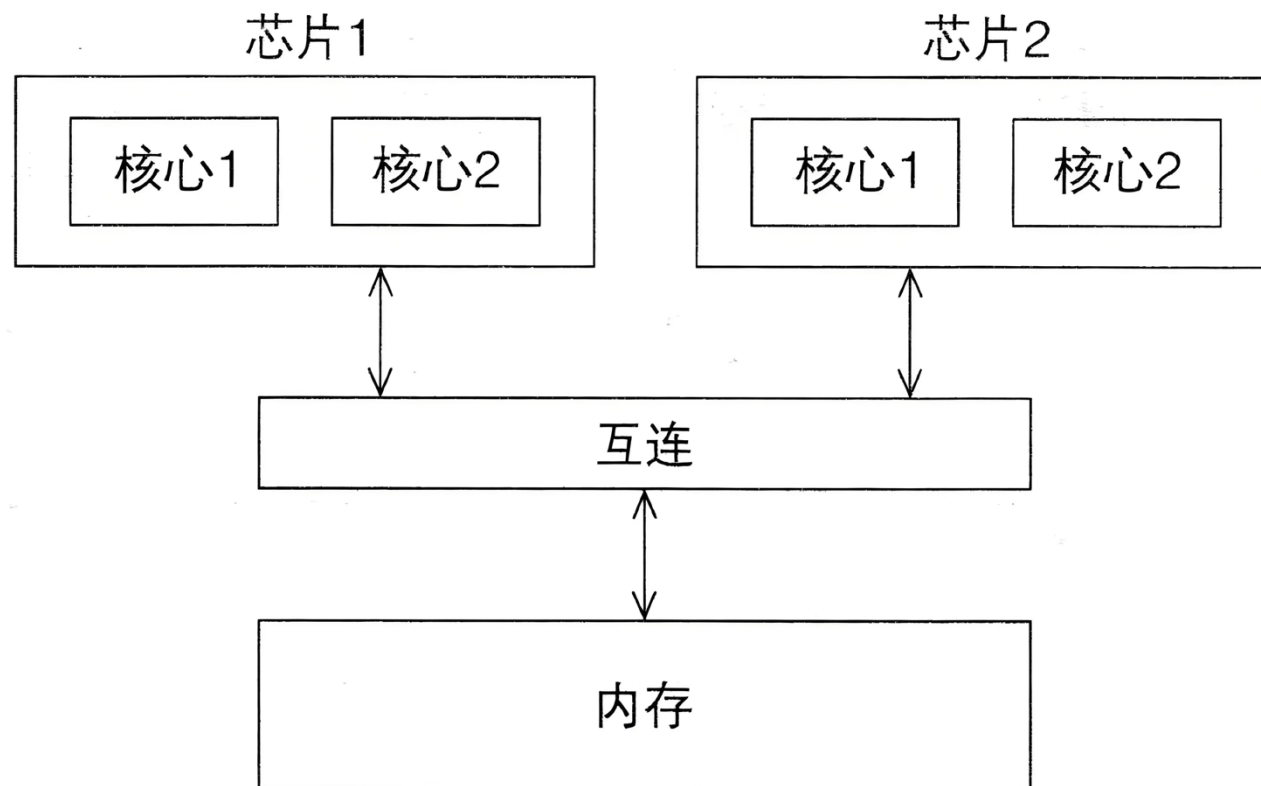
Intel Ultra 9 285K, 24核
24线程, 基础3.7GHz,
最大睿频5.5GHz



AMD锐龙 7 9850X3D, 8
核16线程, 基础4.7GHz,
加速频率5.6GHz

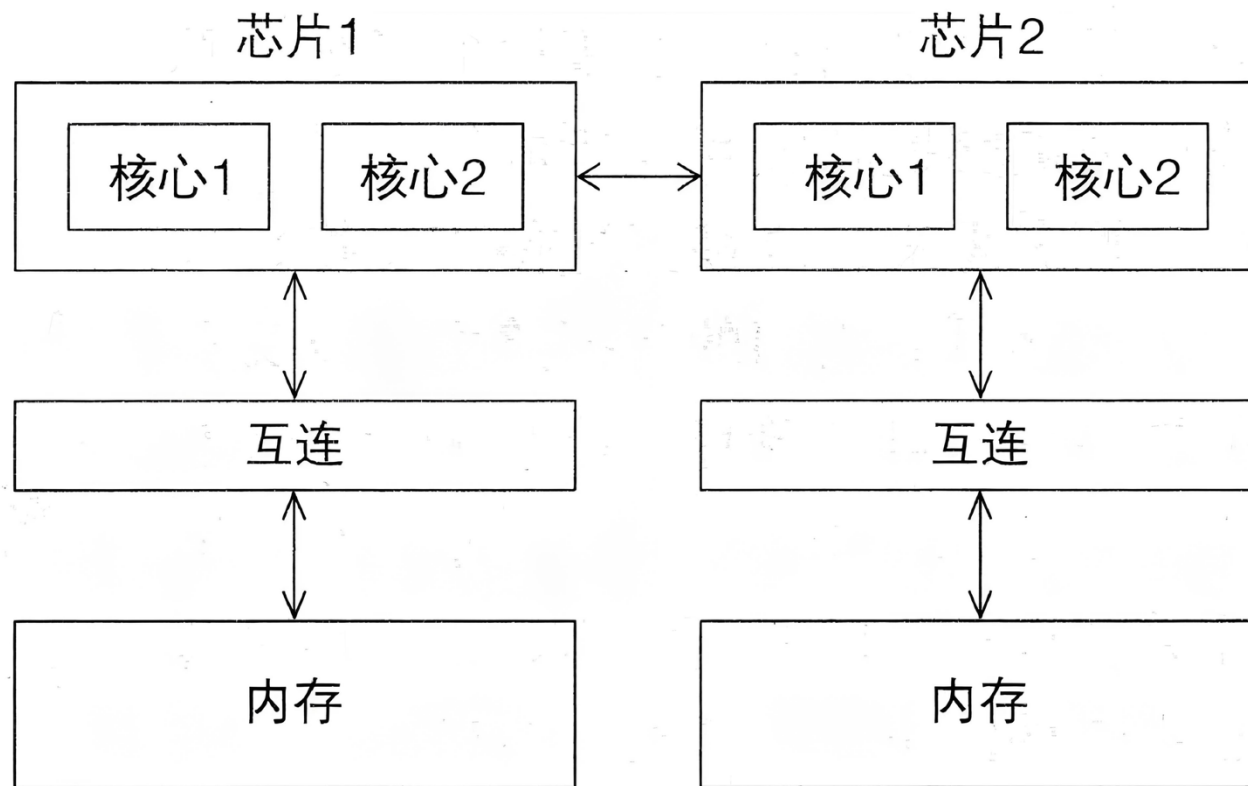
UMA多核心系统

- 所有核心访问所有内存位置的时间都是相同的。



NUMA多核心系统

- 核心直接连接到的内存位置比必须通过另一个芯片访问的内存位置访问速度更快。



分布式内存系统

■ 集群

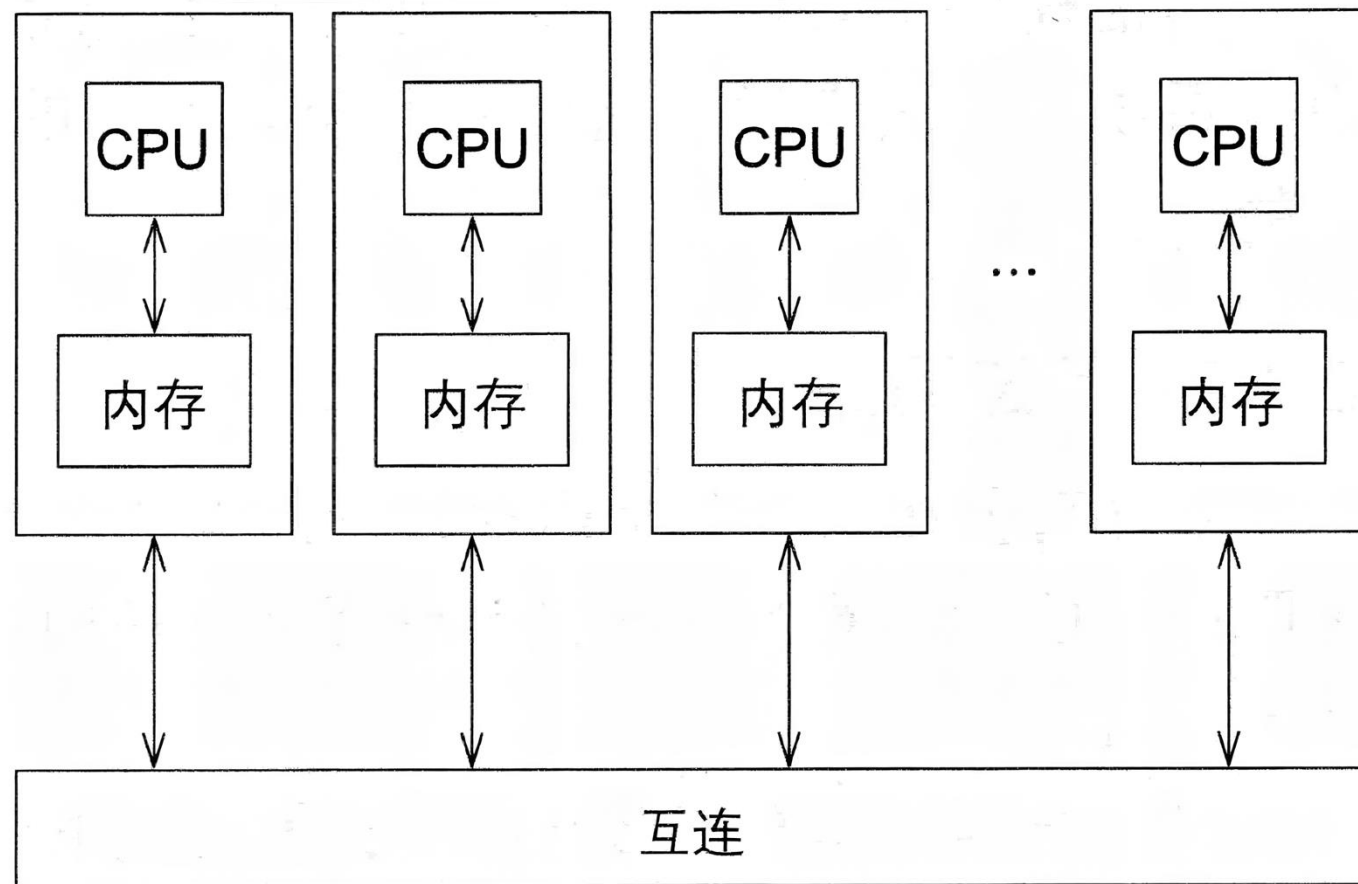
- ▶ 由一系列商用系统组成。
- ▶ 通过商用互连网络（例如以太网）连接。

■ 集群的节点

- ▶ 通过通信网络连接在一起的各个计算单元。
- ▶ 通常是具有一个或多个多核处理器的共享内存系统。



分布式内存系统

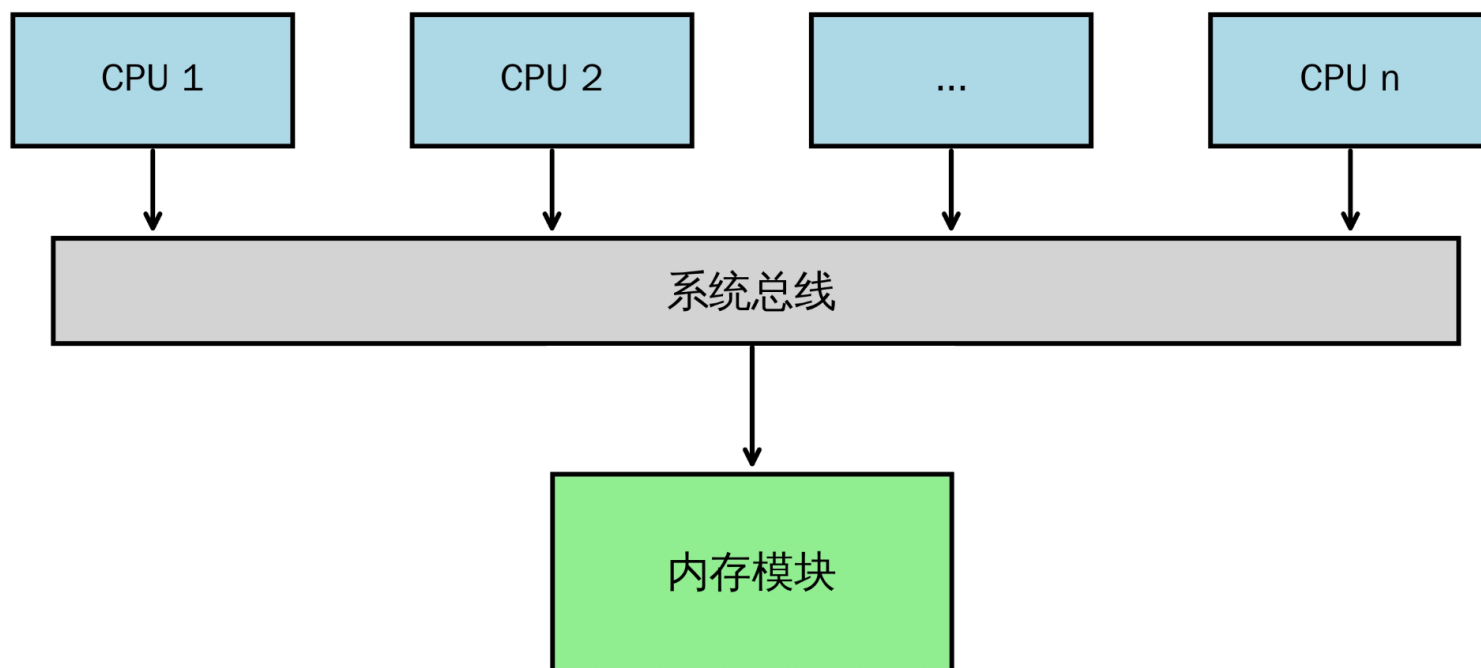


互连网络

- 对分布式和共享内存系统的性能起着决定性的作用。
- 两类：
 - 共享内存的互连网络
 - 分布式内存的互连网络

共享内存的互连网络

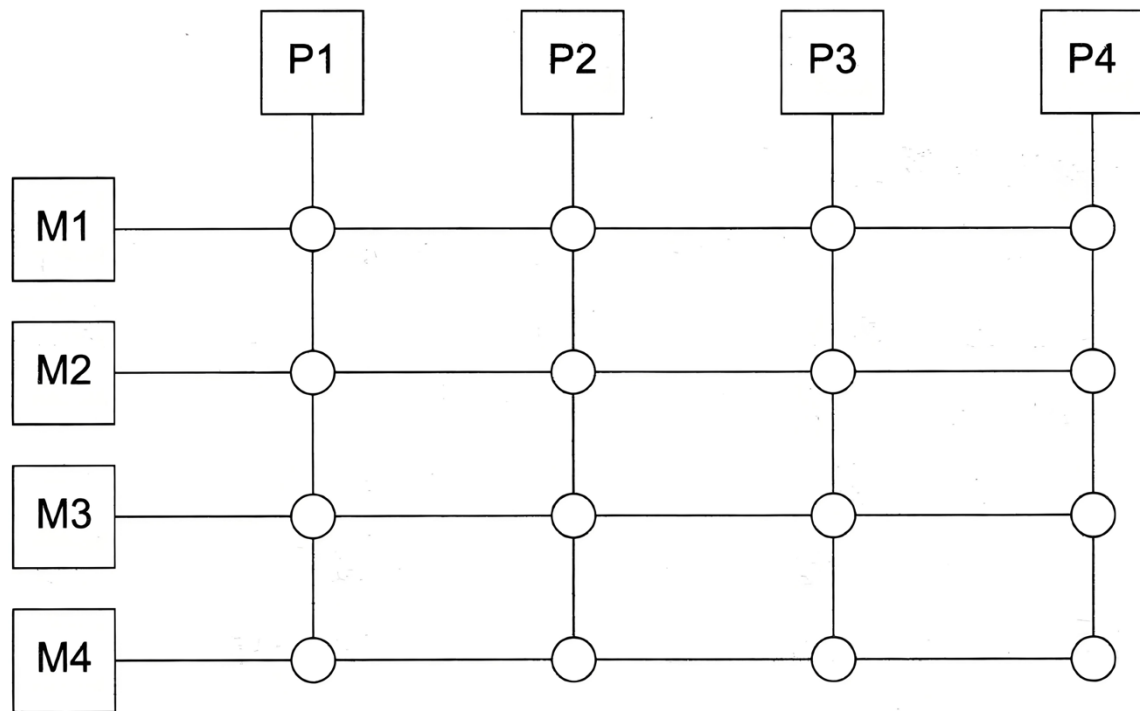
- 总线 - 一组由多条导线组成、可被多个部件分时共享的传输线路。
 - ▶ 共享性：同一时刻仅一个设备可发送信号，多个设备可同时接收。
 - ▶ 分时复用：通过时间分片，让多设备轮流使用总线。
 - ▶ 标准化：接口、信号、速率遵循统一标准（如 PCIe、USB），保障兼容性。



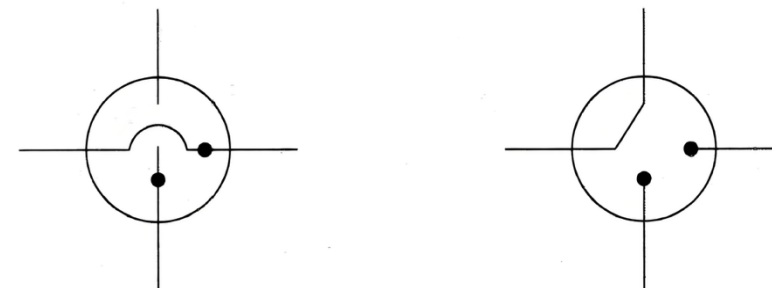
共享内存的互连网络

■ 交换互连

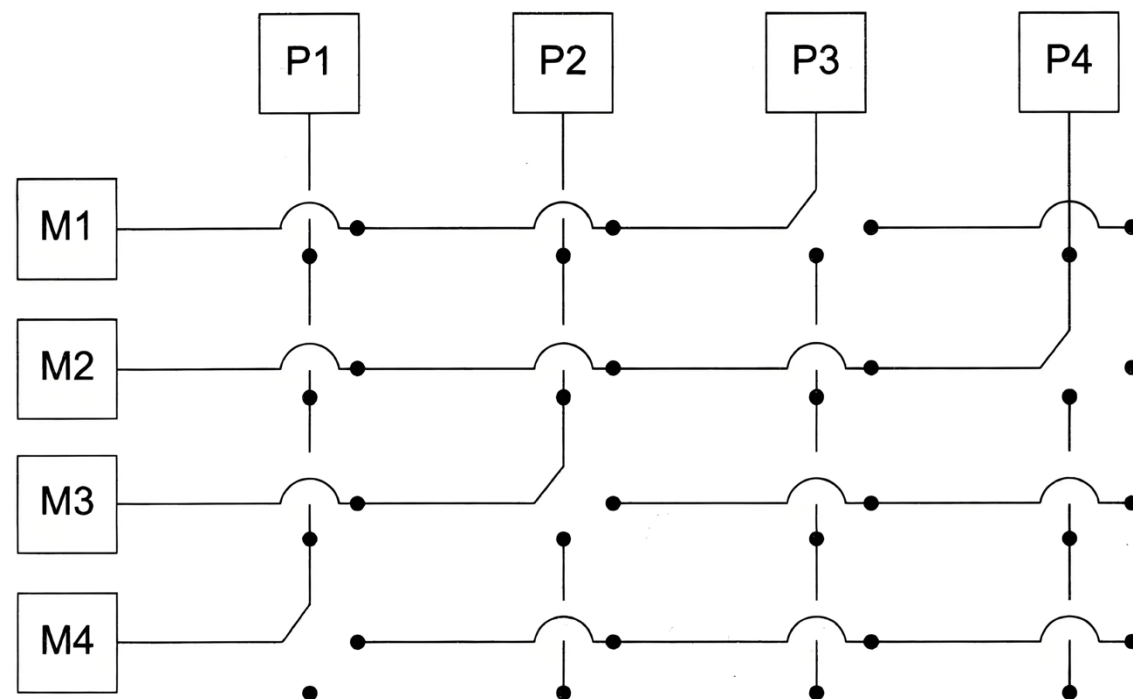
- ▶ 使用交换机来控制连接设备之间的数据路由。
- ▶ 交叉开关
 - 允许不同设备之间同时通信。
 - 比总线快得多。
 - 但是交换机和链路的成本相对较高。



一个交叉开关连四个处理器
(P_i) 和四个内存模块 (M_j)



配置内部交叉开关



处理器同时进行内存访问

分布式内存的互连网络

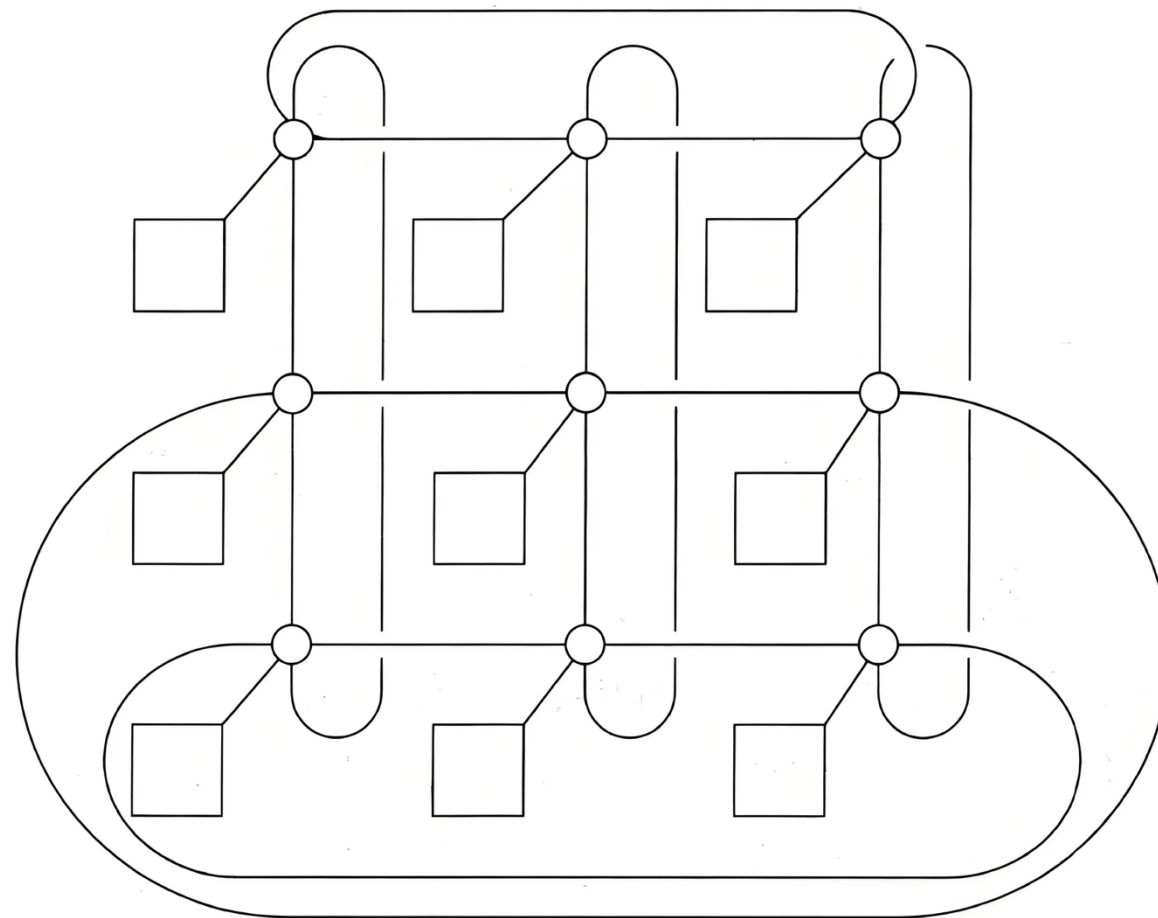
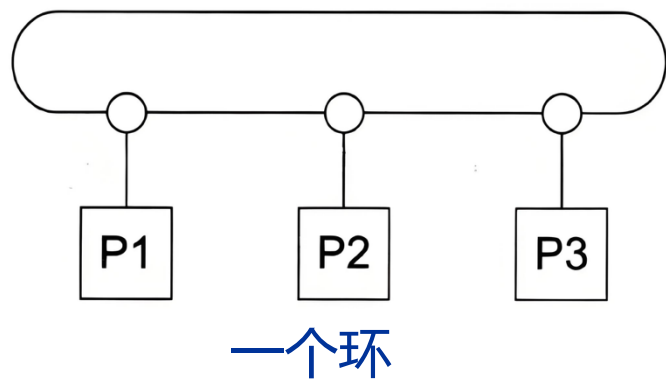
■ 直接互连

- ▶ 每个处理单元 / 节点只和相邻节点直接相连。
- ▶ 没有独立的交换节点，通信链路直接连在节点之间。
- ▶ 路径由源节点到目标节点的跳数决定。

■ 间接互连

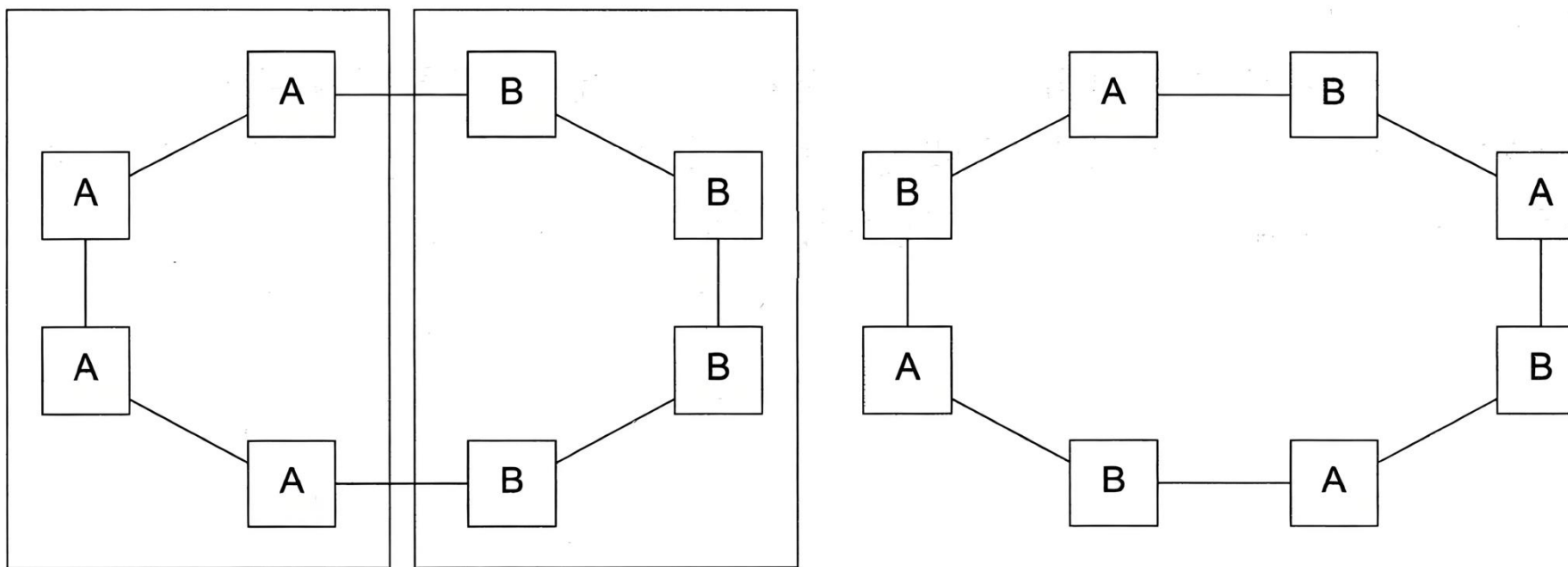
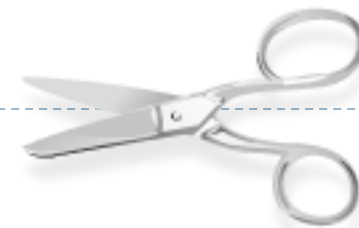
- ▶ 处理节点和交换节点是分开的。
- ▶ 节点之间不直接相连，必须经过交换机/路由器转发。
- ▶ 通信路径由交换开关的状态决定。

直接互连



等分宽度

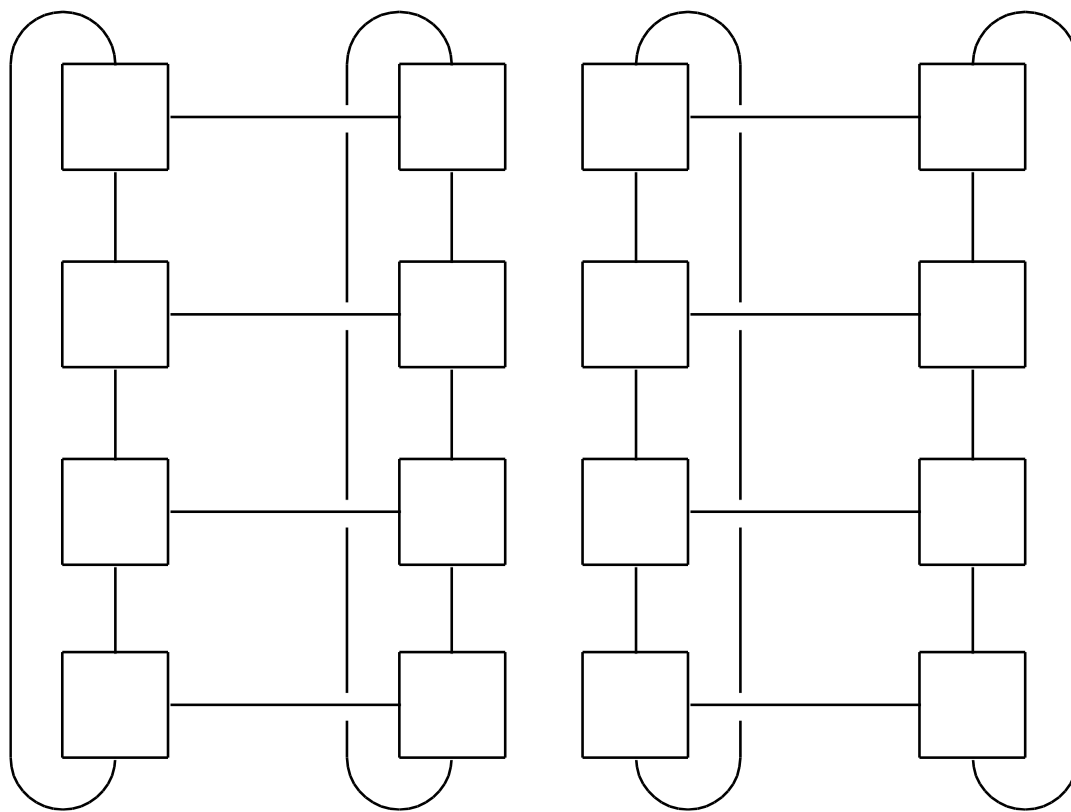
- 同时通信数量或连接性的衡量指标。
- 两半之间跨越鸿沟可以同时进行多少次通信？



一个环的两个分组：a) 两半之间只能进行两次通信，
b) 四个连接可以同时进行

一个二维环形网格的两半

- 将节点集拆分为两个相等部分所需删除的**最小**链接数。



定义

■ 带宽

- ▶ 链路传输数据的速率。
- ▶ 通常以每秒兆比特或兆字节为单位。

■ 等分宽度

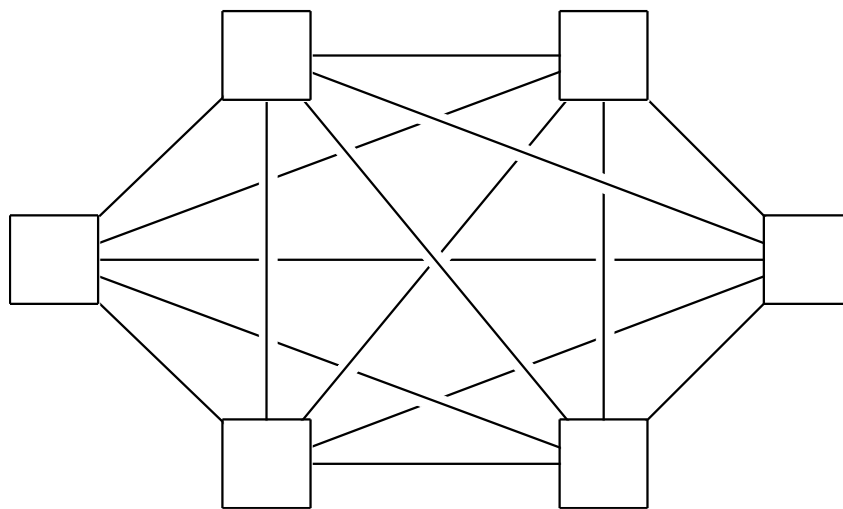
- ▶ 衡量网络质量的指标。
- ▶ 不对连接两端的链路数量进行计算，而是对链路的带宽求和。

全连接网络

- 每个交换机直接连接到其他交换机。

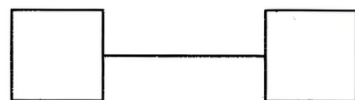
不切实际

等分宽度 = $p^2/4$

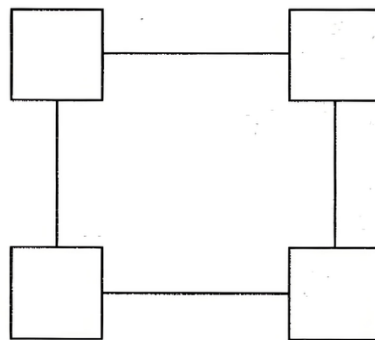


超立方体

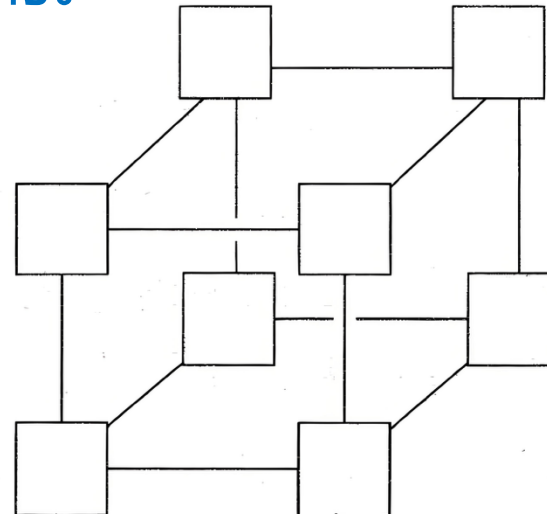
- 高度连接的直接互连。
- 归纳式构建：
 - ▶ 一维超立方体是一个具有两个处理器的完全连接系统；
 - ▶ 二维超立方体是由两个一维超立方体通过连接对应交换机而构建的；
 - ▶ 三维超立方体是由两个二维超立方体构成的。



一维超立方体



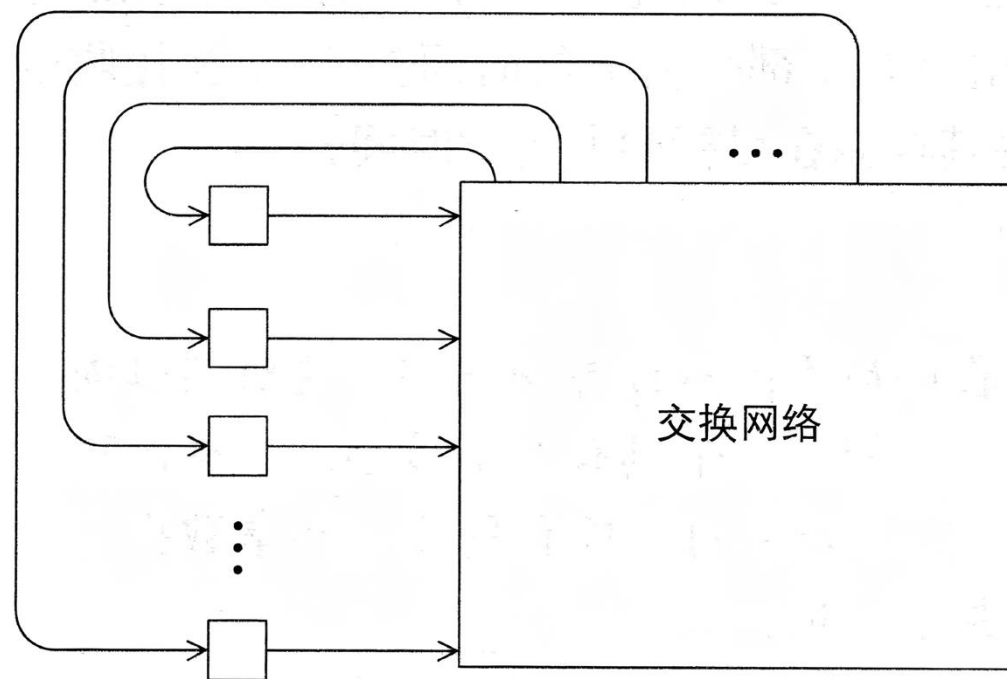
二维超立方体



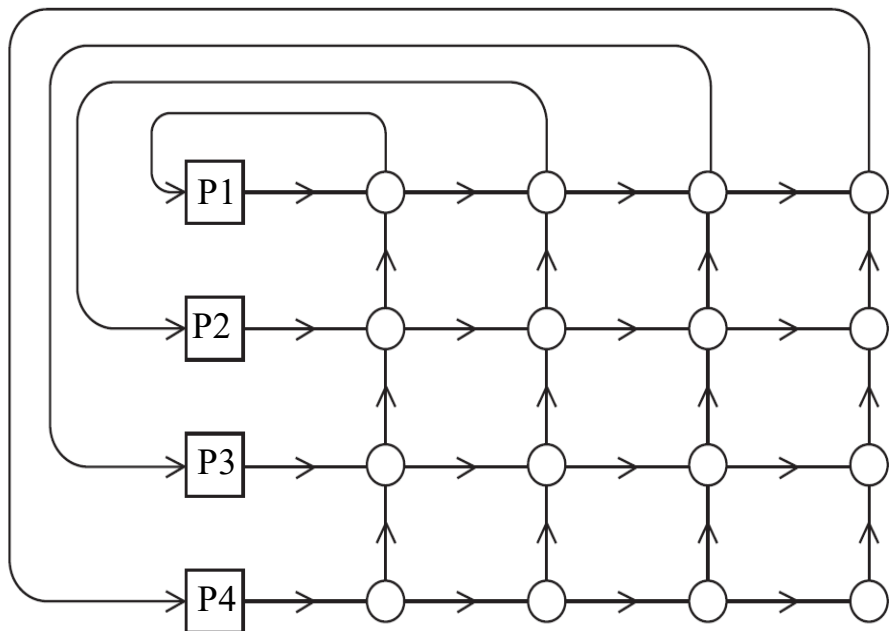
三维超立方体

间接互连

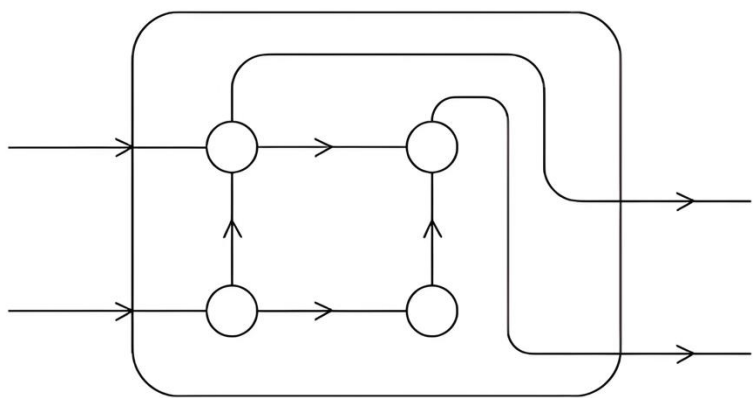
- 间接网络的简单示例：
 - 交叉开关矩阵
 - omega网络
- 通常可表示为单向链路和一组处理器，每个处理器都有一个传出链接和一个传入链接，以及一个交换网络。



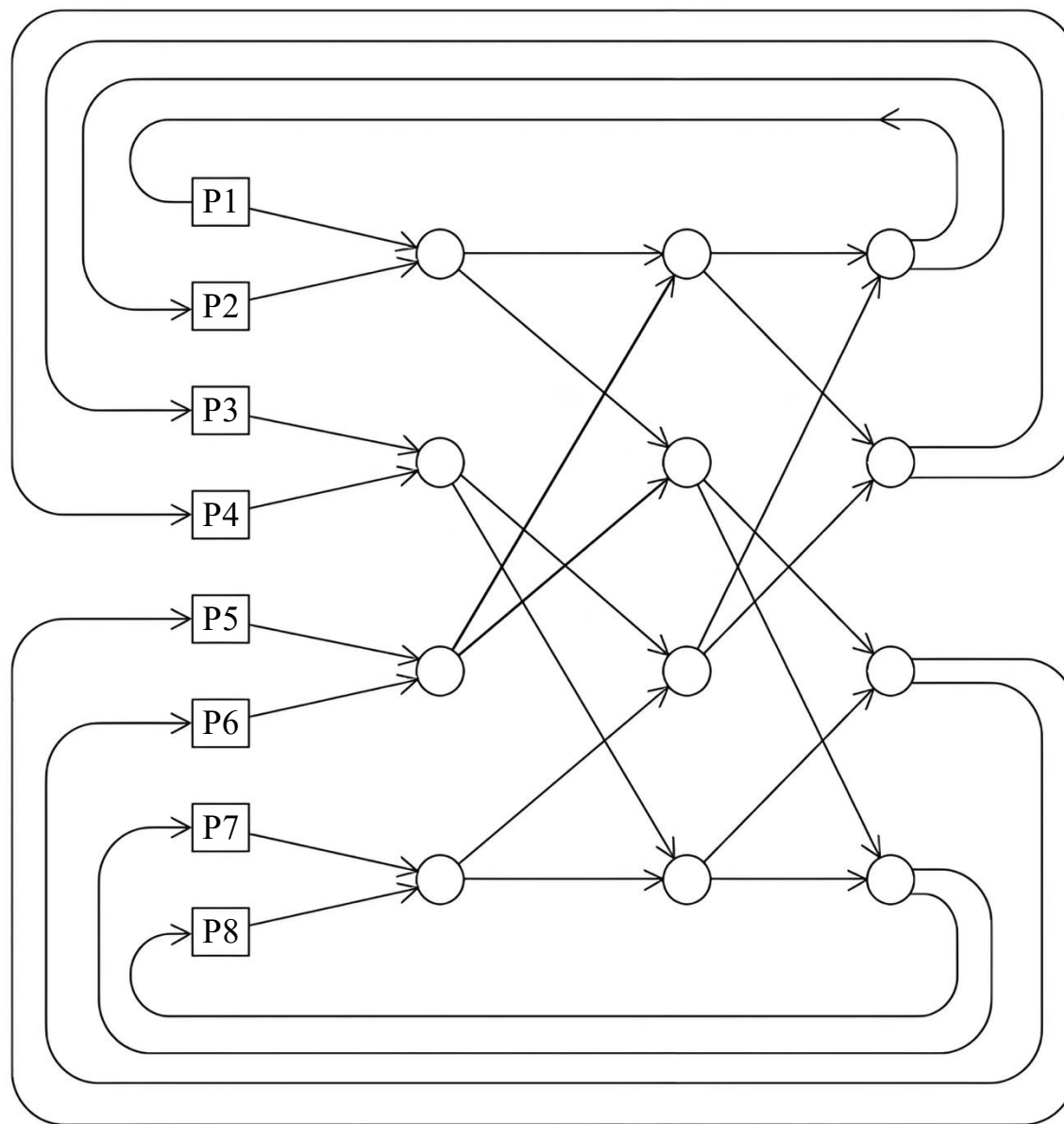
一个通用的间接网络



分布式内存的交叉互连



omega网络中的开关



omega网络

更多定义

- 无论何时传输数据，我们都关心数据到达目的地需要多长时间。
- 延迟
 - ▶ 从源开始传输数据，到目标开始接收第一个字节之间经过的时间。
- 带宽
 - ▶ 目标开始接收第一个字节后接收数据的速率。

$$\text{消息传输时间} = l + n / b$$

延迟 消息长度 带宽

Cache一致性

- CPU缓存是由系统硬件管理的，程序员不能直接控制它们。

y0由核心0私有

y1和z1由核心1私有

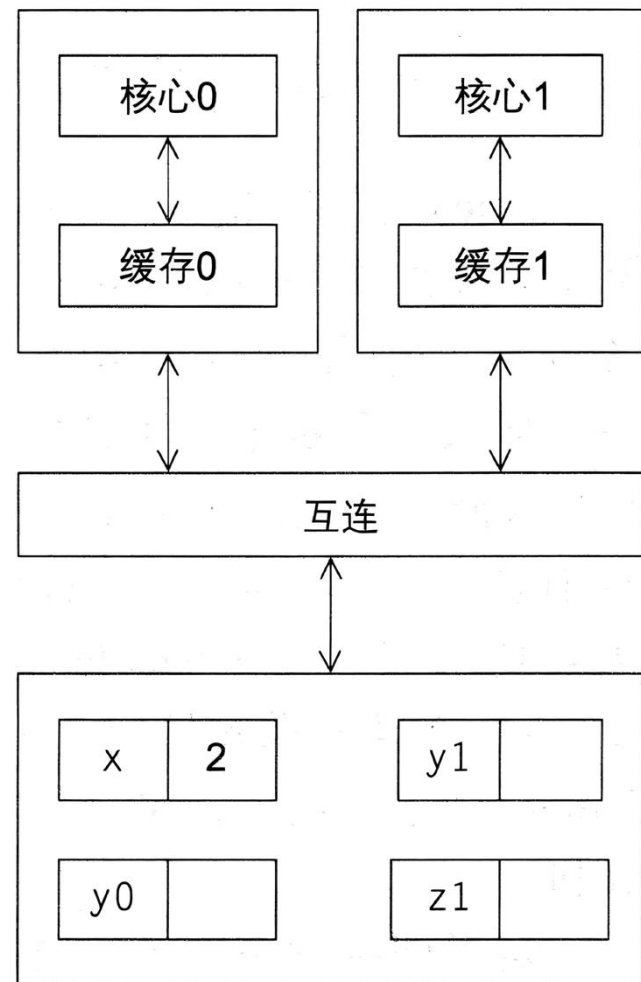
x = 2; /*共享变量*/

时刻	核心 0	核心 1
0	y0 = x;	y1 = 3*x;
1	x = 7;	不涉及 x 的语句
2	不涉及 x 的语句	z1 = 4*x;

y0 = 2

y1 = 6

z1 = ???



具有两个核心和两个缓存的共享内存系统

监听缓存一致性

- 核心共享一条总线。
- 所有连接到总线的核心都可以看到总线上传输的任何信号。
- 当核心0更新其缓存中存储的x副本时，也通过总线广播此信息。
- 如果核心1正在监听总线，它将看到x已经更新，并且它可以将其x副本标记为无效。

基于目录的缓存一致性

- 使用被称为目录的数据结构来存储每个缓存行的状态。
- 当一个变量被更新时，先查询目录，然后让所有缓存了该变量所在缓存行的核心，将其缓存副本置为无效。

并行软件



重担落在了软件上

- 硬件与编译器能够跟上所需的步伐。
- 从现在起.....
 - ▶ 在共享内存程序中：
 - 启动一个进程并派生出多个线程。
 - 线程执行任务。
 - ▶ 在分布式内存程序中：
 - 启动多个进程。
 - 进程执行任务。

SPMD - 单程序多数据

- SPMD程序由一个单一的可执行文件组成，通过使用条件分支，表现得像多个不同的程序一样。

```
if (I'm thread process i)  
    do this;  
else  
    do that;
```



编写并行程序

- 在进程 / 线程之间分配工作，使：
 - 每个进程 / 线程得到大致相同的工作量。
 - 所需的通信量最小化。
- 安排进程 / 线程进行同步。
- 安排进程 / 线程之间的通信。

```
double x[n], y[n];  
.  
.  
.  
for (int i = 0; i < n; i++)  
    x[i] += y[i];
```

共享内存

■ 动态线程

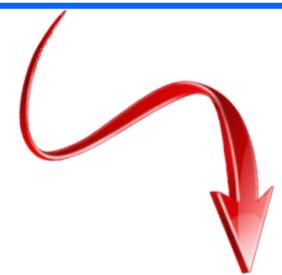
- ▶ 主线程等待工作，创建新线程执行，当完成工作后，便终止运行。
- ▶ 资源利用效率高，但线程的创建和终止耗时较长。

■ 静态线程

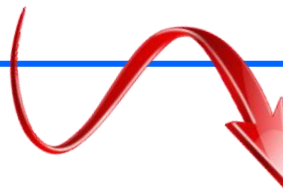
- ▶ 创建线程池并为其分配工作，直到所有工作完成后，所有线程才会停止运行。
 -
- ▶ 性能更优，但可能浪费系统资源。

非不确定性

```
...  
printf ( "Thread %d > my_val = %d\n" , my_rank, my_x ) ;  
...
```



Thread 1 > my_val = 19
Thread 0 > my_val = 7



Thread 0 > my_val = 7
Thread 1 > my_val = 19

非确定性

```
my_val = Compute_val ( my_rank );  
x += my_val ;
```

时刻	核心 0	核心 1
0	完成对 my_val 的赋值	调用 Compute_val
1	将 x = 0 加载到寄存器 核心 1	完成对 my_val 的赋值
2	将 my_val = 7 加载到寄存器	将 x = 0 加载到寄存器
3	将 my_val = 7 添加到 x	将 my_val = 19 加载到寄存器

非确定性

- 竞争条件
- 临界区
- 互斥
- 互斥锁（互斥量，或简称为锁）

```
my_val = Compute_val ( my_rank ) ;  
Lock(&add_my_val_lock ) ;  
x += my_val ;  
Unlock(&add_my_val_lock ) ;
```

忙等待

```
my_val = Compute_val(my_rank);  
if (my_rank == 1)  
    while (!ok_for_1);    /* 忙等待循环 */  
x += my_val;             /* 关键区 */  
if (my_rank == 0)  
    ok_for_1 = true;      /* 使线程1更新x */
```

消息传递

```
char message[100];  
. . .  
my_rank = Get_rank();  
if (my_rank == 1) {  
    sprintf(message, "Greetings from process 1");  
    Send(message, MSG_CHAR, 100, 0);  
} else if (my_rank == 0) {  
    Receive(message, MSG_CHAR, 100, 1);  
    printf("Process 0 > Received: %s\n", message);  
}
```


分区全局地址空间语言

```
shared int n = . . . ;
shared double x[n], y[n];
private int i, my_first_element, my_last_element;
my_first_element = . . . ;
my_last_element = . . . ;

/* 初始化 x 和 y */
. . .

for (i = my_first_element; i <= my_last_element; i++)
    x[i] += y[i];
```

输入和输出

- 在分布式内存程序中，只有进程0将访问stdin。在共享内存程序中，只有主线程或线程0将访问stdin。
- 在分布式内存和共享内存程序中，所有进程 / 线程都可以访问stdout和stderr。
- 然而，由于输出到stdout的顺序不确定，在大多数情况下，只有一个进程 / 线程会将结果输出到stdout。但输出调试程序结果是个例外。在这种情况下，通常会有多个进程 / 线程写入stdout。

输入和输出

- 只有一个进程 / 线程会尝试访问stdin、stdout或stderr以外的任何单个文件。例如，每个进程 / 线程都可以打开自己的私有文件进行读写，但没有两个进程 / 线程会打开同一个文件。
- 调试输出应始终包括生成输出的进程 / 线程的序列号或ID。

性能



加速比和效率

- 核心数量 = p
- 串行运行时间 = T_{serial}
- 并行运行时间 = T_{parallel}

$$\text{加速比 } S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

线性加速比: $S = p$

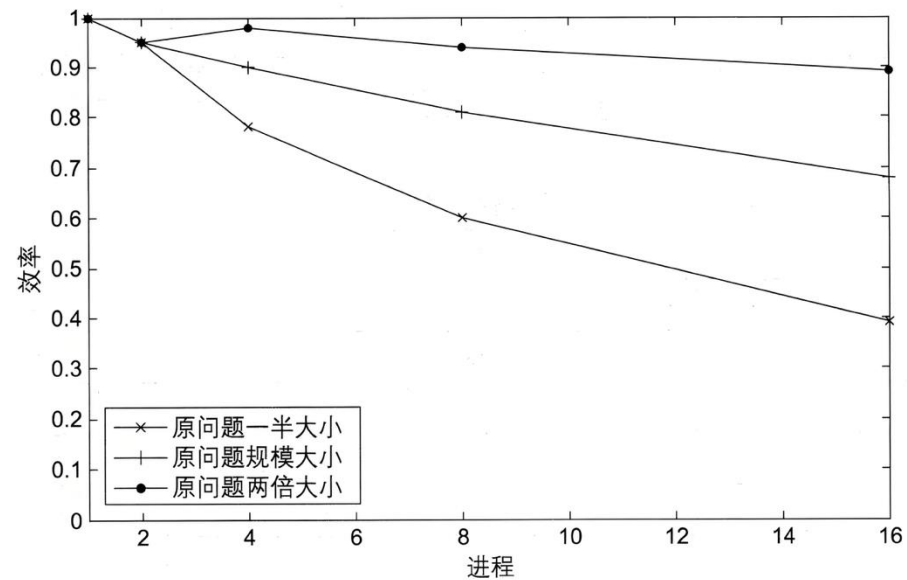
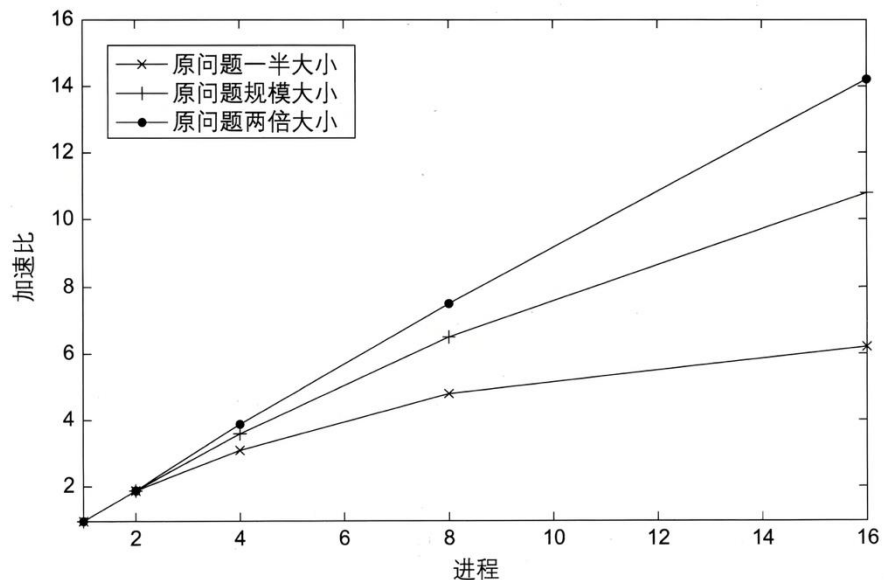
$$\begin{aligned} \text{效率 } E &= \frac{S}{p} = \frac{\left(\frac{T_{\text{serial}}}{T_{\text{parallel}}} \right)}{p} \\ &= \frac{T_{\text{serial}}}{p \cdot T_{\text{parallel}}} \end{aligned}$$



p	1	2	4	8	16
S	1.0	1.9	3.6	6.5	10.8
$E=S/p$	1.0	0.95	0.90	0.81	0.68

不同问题规模的并行政程序的加速比和效率

	p	1	2	4	8	16
原问题一半大小	S	1.0	1.9	3.1	4.8	6.2
	E	1.0	0.95	0.78	0.60	0.39
原问题规模大小	S	1.0	1.9	3.6	6.5	10.8
	E	1.0	0.95	0.90	0.81	0.68
原问题两倍大小	S	1.0	1.9	3.9	7.5	14.2
	E	1.0	0.95	0.98	0.94	0.89



开销的影响

$$T_{\text{parallel}} = T_{\text{serial}}/p + T_{\text{overhead}}$$

阿姆达定律

- 除非几乎所有串行程序都并行化，否则无论可用的核心数量如何，可能的加速都将非常有限。



示例

- 假设能够并行化一个串行程序的90%。
- 假设并行化是完美的，也就是说，无论使用的核心数是多少，程序这部分的加速比都是 p 。

- 串行运行时间 $T_{\text{serial}}=20$ 。

- 并行部分的运行时间：

$$0.9 \times T_{\text{serial}} / p = 18 / p$$

- 未并行部分的运行时间：

$$0.1 \times T_{\text{serial}} = 2$$

```
void main()
{
    int i, m;
    double x, pi, sum, step;
    m = 1000;
    sum = 0;
    step = 1.0 / m;
    for (i = 0; i < m; i++)
    {
        x = (i + 0.5) * step;
        sum += 4 / (1 + x * x);
    }
    pi = step * sum;
}
```

示例

■ 总体并行运行时间:

$$T_{\text{parallel}} = 0.9 \times T_{\text{serial}} / p + 0.1 \times T_{\text{serial}} = 18 / p + 2$$

■ 加速比:

$$S = \frac{T_{\text{serial}}}{0.9 \times T_{\text{serial}} / p + 0.1 \times T_{\text{serial}}} = \frac{20}{18 / p + 2}$$

可扩展性

- 一般来说，如果一个问题能够处理不断增大的规模，那么它就是**可扩展的**。
- 如果增加进程 / 线程的数量时，可以在不增加问题规模的情况下保持固定的效率，那么该程序就可以说是具有**强可扩展性**。
- 如果我们可以通过与增加进程 / 线程数量相同的速度增加问题规模来保持固定的效率，那么该程序被认为是**弱可扩展性**。

计时

- 什么是时间？
- 从开始到结束？
- 所关注的程序片段？
- CPU时间？
- 挂钟时间？



计时

- Get_current_time返回自过去某个固定时间以来经过的秒数。

```
private double start, finish;
```

```
. . .
```

```
start = Get_current_time();
```

```
/* 我们希望计时的代码 */
```

```
. . .
```

```
finish = Get_current_time();
```

```
printf("The elapsed time = %e seconds\n", finish-start);
```

假设函数

MPI_Wtime

omp_get_wtime

计时

```
shared double global_elapsed;  
private double my_start, my_finish, my_elapsed;  
. . .  
/* 所有进程/线程同步 */  
Barrier();  
my_start = Get_current_time();  
  
/* 我们希望计时的代码 */  
. . .  
  
my_finish = Get_current_time();  
my_elapsed = my_finish - my_start;  
  
/* 查找所有进程/线程的最大值 */  
global_elapsed = Global_max(my_elapsed);  
if (my_rank == 0)  
    printf("The elapsed time = %e seconds\n",  
           global_elapsed);
```

并行程序设计



Foster方法论

- **划分** - 将要执行的计算和计算操作的数据划分为小任务。
 - ▶ 这里的重点应该是确定可以并行执行的任务。
- **通信** - 确定在上一步中确定的任务之间需要进行哪些通信。
- **凝聚或聚合** - 将第1步中确定的任务和通信合并到更大的任务中。
 - ▶ 例如，如果必须先执行任务A，然后才能执行任务B，那么可以将它们聚合到单一的复合任务中。
- **映射** - 将上一步中标识的复合任务分配给进程 / 线程。
 - ▶ 这样做是为了使通信最小化，并且每个进程 / 线程得到大致相同的工作量。

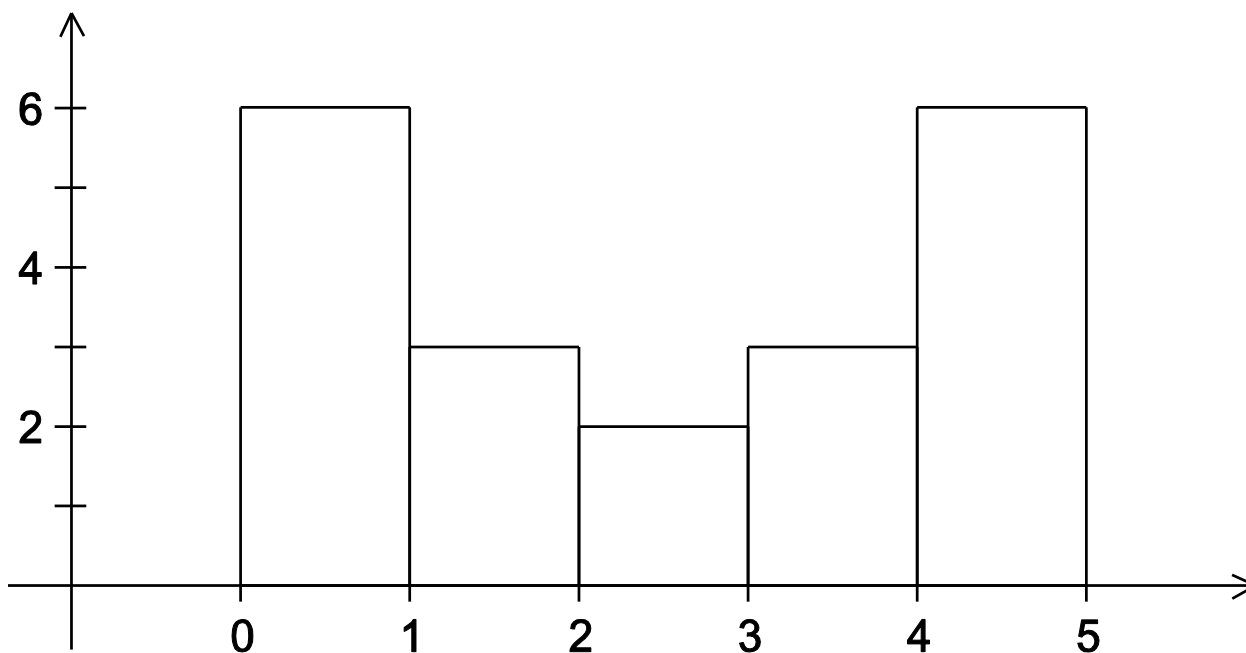
示例

- 以 1000×1000 矩阵 $\times$ 向量为例，4 线程并行。

步骤	操作	效果
Partitioning	拆分为 10^6 次独立乘法（细粒度）	暴露最大并行度
Communication	每行乘法结果需归约求和	定义行内通信
Agglomeration	合并为4个行块任务（粗粒度）	消除行内通信，任务数 = 线程数
Mapping	每个行块任务分配给1个线程	负载均衡，无跨线程通信

示例 - 柱状图

1.3, 2.9, 0.4, 0.3, 1.3, 4.4, 1.7, 0.4, 3.2, 0.3,
4.9, 2.4, 3.1, 4.4, 3.9, 0.4, 4.2, 4.5, 4.9, 0.9



串程序

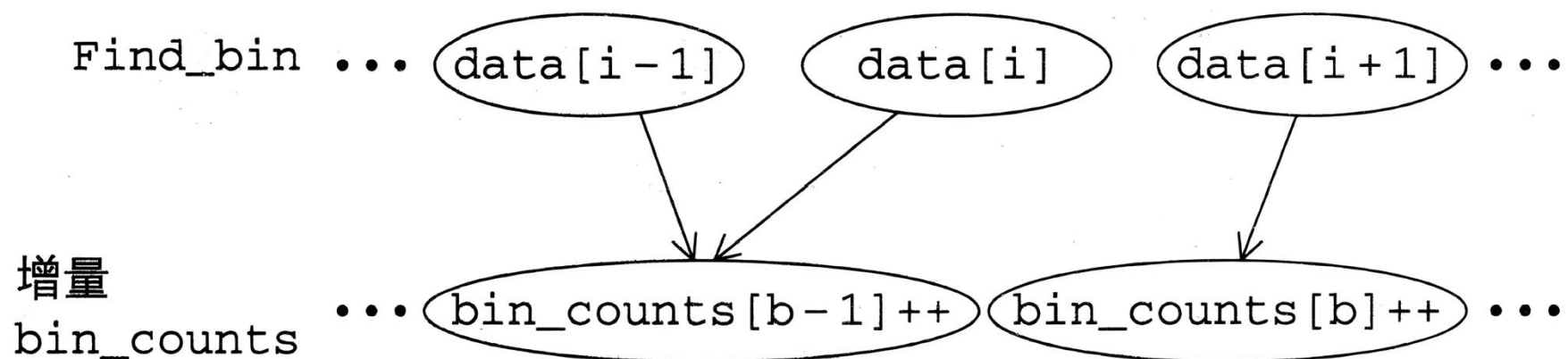
■ 输入：

- ▶ 测量次数, `data_count`。
- ▶ `data_count`个浮点数的数组, `data`。
- ▶ 包含最小值的桶的最小值, `min_meas`。
- ▶ 包含最大值的桶的最大值, `max_meas`。
- ▶ 桶的数量, `bin_count`。

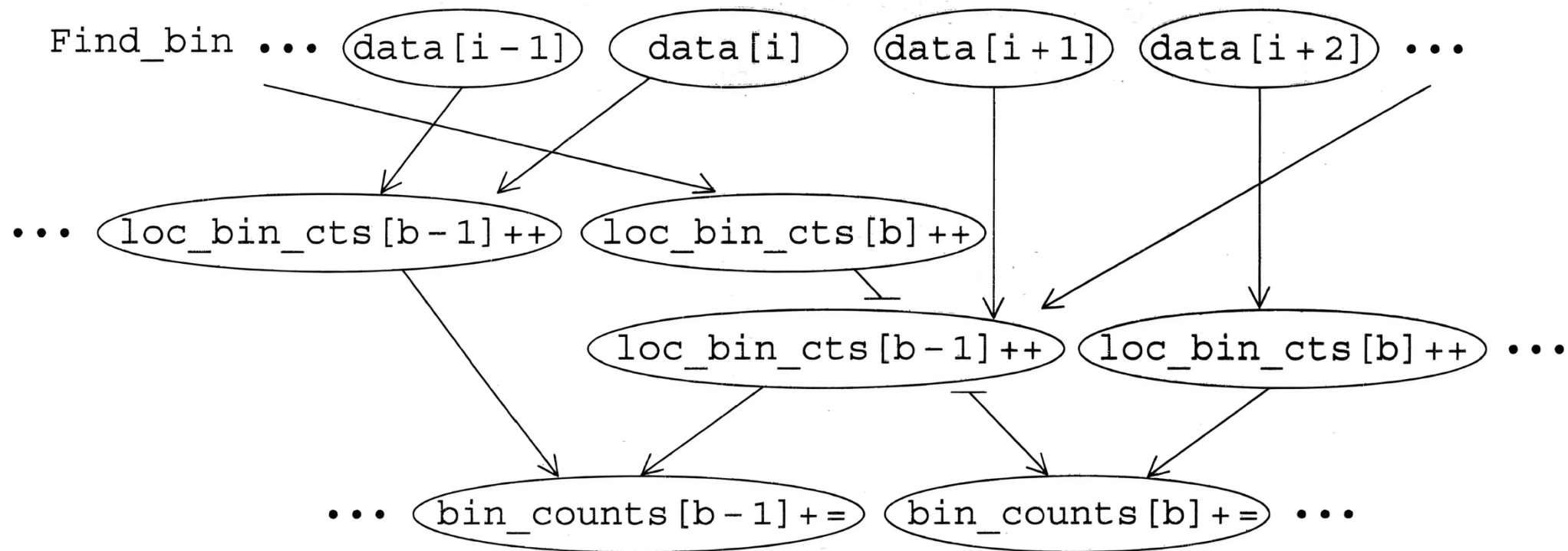
■ 输出：

- ▶ `bin_count`个浮点数的数组`bin_maxes`。
- ▶ `bin_count`个整数的数组`bin_counts`。

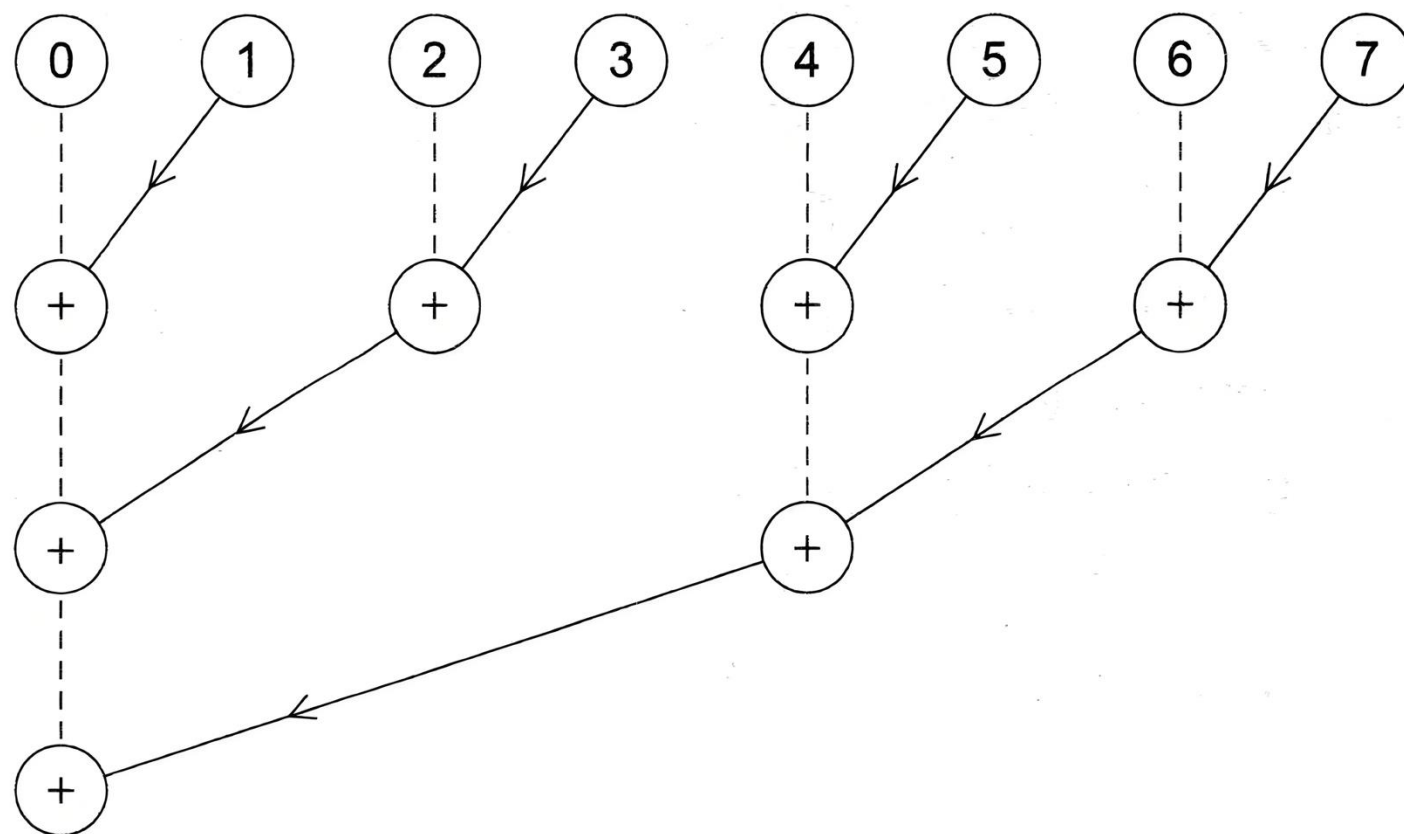
Foster方法论的前两个阶段



任务与通信的另一种定义



累加本地数组



小结

■ 串行系统

- ▶ 计算机硬件的标准模型是冯·诺依曼体系结构。

■ 并行硬件

- ▶ Flynn分类法。

■ 并行软件

- ▶ 着重于同构MIMD系统以及具有CPU和GPU的异构系统的软件，此类系统的大多数程序都由单个程序组成，该程序通过在线程 / 进程和 / 或分支之间划分数据来获得并行性。
- ▶ SPMD程序。

小结

■ 输入和输出

- ▶ 将针对同构MIMD系统的程序编写程序：其中一个进程或线程可以访问stdin，所有进程都可以访问stdout和stderr。
- ▶ 然而，由于除调试输出外的不确定性，通常会有一个进程或线程访问标准输出。

小结

■ 性能

- ▶ 加速比
- ▶ 效率
- ▶ 阿姆达定律
- ▶ 可扩展性

■ 并行程序设计

- ▶ Foster方法论